

Predicting Insurance Claim Severity

1. Introduction

Background

Insurance companies operate in a highly competitive environment where **accurate risk assessment** is critical for profitability. Claim severity prediction helps insurers:

- **Set fair premiums** based on individual risk profiles.
- **Mitigate financial losses** by identifying high-risk policyholders.
- **Improve customer satisfaction** through personalized pricing.

Problem Statement

The dataset contains **10,302 insurance claims** with 27 features, including:

- **Policyholder demographics** (age, income, occupation).
- **Vehicle details** (car type, age, value).
- **Driving history** (past claims, traffic violations).

Goal: Predict **claim severity (CLM_AMT)** to optimize pricing and risk management.

Objectives

1. **Clean and preprocess** data for analysis.
2. **Explore relationships** between claim amounts and predictors.
3. **Build predictive models** and evaluate performance.
4. **Provide actionable insights** for business strategy.

Data Preprocessing & Cleaning

Sample of dataset.

df.sample(5)

	ID	KIDSDRIV	BIRTH	AGE	HOMEKIDS	YOJ	INCOME	PARENT1	HOME_VAL	MSTATUS	...	CAR_TYPE	RED_CAR	OLDCLAIM	CLM_FREQ
8250	5919898	0	30NOV50	48.0	1	12.0	\$80,016	Yes	\$0	z_No	...	Van	no	\$0	0
5996	130933470	0	25JAN43	56.0	0	14.0	NaN	No	\$238,812	Yes	...	Minivan	no	\$0	0
8015	874176163	0	22SEP50	49.0	0	2.0	\$132,359	No	\$341,357	Yes	...	Panel Truck	no	\$3,408	1
10154	340299187	0	20AUG51	48.0	0	13.0	\$122,643	No	\$315,375	Yes	...	Sports Car	no	\$47,343	2
8358	204021526	0	18JUL51	48.0	0	10.0	\$34,606	No	\$163,682	Yes	...	Minivan	yes	\$0	0

5 rows × 27 columns

df.sample(5)

PARENT1	HOME_VAL	MSTATUS	...	CAR_TYPE	RED_CAR	OLDCLAIM	CLM_FREQ	REVOKED	MVR_PTS	CLM_AMT	CAR_AGE	CLAIM_FLAG	URBANICITY
Yes	\$0	z_No	...	Van	no	\$0	0	No	0	\$4,043	7.0	1	Highly Urban/ Urban
No	\$238,812	Yes	...	Minivan	no	\$0	0	No	0	\$0	15.0	0	Highly Urban/ Urban
No	\$341,357	Yes	...	Panel Truck	no	\$3,408	1	No	5	\$0	15.0	0	Highly Urban/ Urban
No	\$315,375	Yes	...	Sports Car	no	\$47,343	2	No	4	\$0	16.0	0	Highly Urban/ Urban
No	\$163,682	Yes	...	Minivan	yes	\$0	0	No	1	\$3,278	1.0	1	Highly Urban/ Urban

Variables and Explanation

Variable	Explanation
ID	Unique identifier for each customer.
KIDSDRIV	Number of kids the customer has who are of driving age.
BIRTH	Date of birth of the customer.
AGE	Age of the customer.
HOMEKIDS	Number of kids the customer has at home.
YOJ	Years on the job (number of years the customer has been employed).
INCOME	Income of the customer.
PARENT1	Indicates whether the customer is a single parent (Yes/No).
HOME_VAL	Value of the customer's home.
MSTATUS	Marital status of the customer (Yes/No).
GENDER	Gender of the customer (M/F).
EDUCATION	Education level of the customer.
OCCUPATION	Occupation of the customer.
TRAVTIME	Daily travel time to work.
CAR_USE	Purpose of car usage (Private/Commercial).
BLUEBOOK	Value of the customer's car.
TIF	Time in force (Years).
CAR_TYPE	Type of car.
RED_CAR	Indicates whether the customer owns a red car (yes/no).
OLDCLAIM	Total amount of previous claims.
CLM_FREQ	Number of claims reported.
REVOKED	Indicates whether the customer's driver's license has been revoked (Yes/No).
MVR_PTS	Number of motor vehicle record points.
CLM_AMT	Possible future claims.
CAR_AGE	Age of the customer's car.
CLAIM_FLAG	Indicates whether a claim was filed (1/0).
URBANICITY	Urbanicity type ((Highly Urban/Urban)/(Highly Rural/Rural)).

Handling Missing Data

```
df.isnull().sum()
```

```
ID          0
KIDSDRIV    0
BIRTH       0
AGE         7
HOMEKIDS    0
YOJ         548
INCOME      570
PARENT1     0
HOME_VAL    575
MSTATUS     0
GENDER      0
EDUCATION   0
OCCUPATION  665
TRAVTIME    0
CAR_USE     0
BLUEBOOK   0
TIF         0
CAR_TYPE    0
RED_CAR     0
OLDCLAIM    0
CLM_FREQ    0
REVOKED     0
MVR_PTS     0
CLM_AMT     0
CAR_AGE     639
CLAIM_FLAG  0
URBANICITY  0
dtype: int64
```

Missing values can bias results. So I addressed them as follow:

Variable with the missing values and Strategy

- Age values missing 7 filled with median(44.8 years)
- Year of job 548 missing values filled median imputation 11 years
- Income 570 missing Converted from text to float and filled with median 61,127
- Occupation 665 filled with labelled as “Unknown”
- Car age 639 missing filled with median of 8 years

Why median over mean?

Income and age are skewed; medians are less affected by outliers.

Data Type Conversions

Currency fields (INCOME, HOME_VAL, CLM_AMT, BLUEBOOK, OLDCLAIM) were stored as text (e.g., "\$50K"). We:

- Removed \$ and commas.
- Converted to numeric values for calculations.

Stats of data

```
df.describe()
```

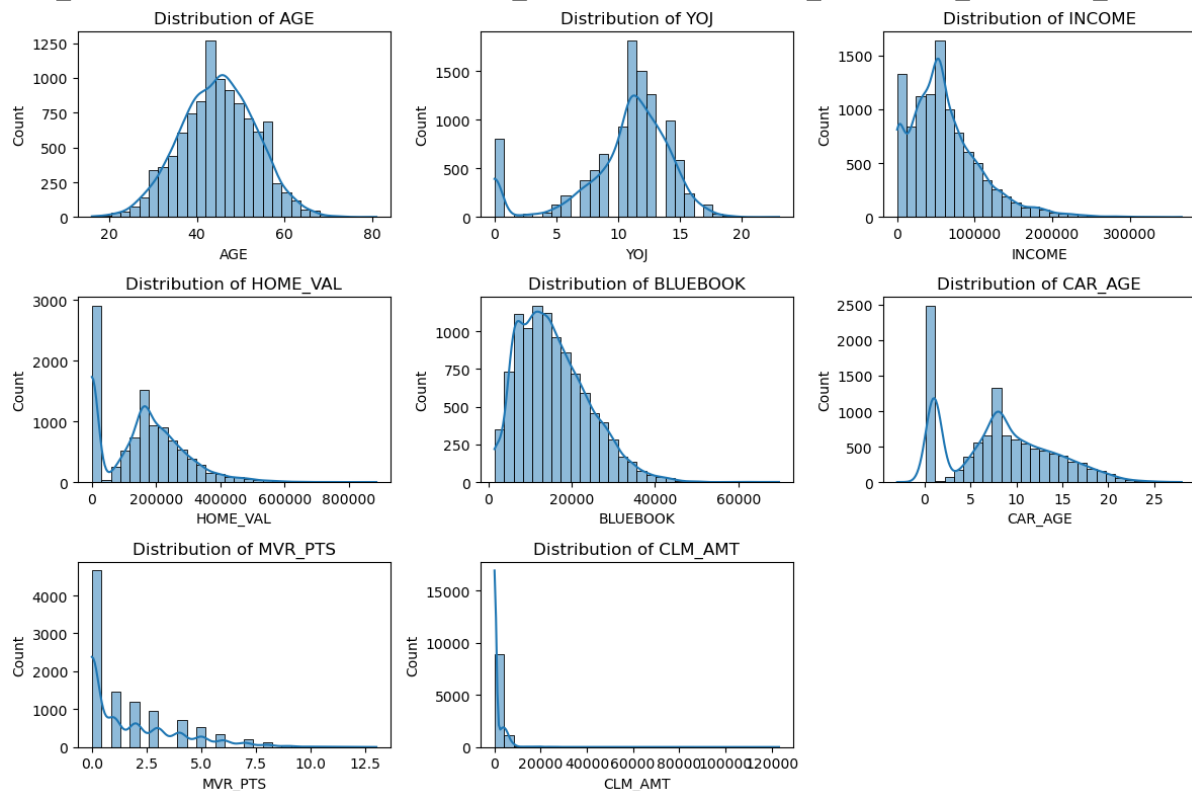
	ID	KIDSDRIV	AGE	HOMEKIDS	YOJ	INCOME	HOME_VAL	TRAVTIME	BLUEBOOK	TIF	OLD
count	1.030200e+04	10302.000000	10302.000000	10302.000000	10302.000000	10302.000000	10302.000000	10302.000000	10302.000000	10302.000000	10302.000000
mean	4.956631e+08	0.169288	44.837507	0.720443	10.502038	61127.057562	154865.606387	33.416424	15659.923316	5.329159	4033.1
std	2.864675e+08	0.506512	8.603521	1.116323	3.999896	46162.151534	125538.938281	15.869687	8428.765264	4.110795	8733.1
min	6.317500e+04	0.000000	16.000000	0.000000	0.000000	0.000000	0.000000	5.000000	1500.000000	1.000000	0.000000
25%	2.442869e+08	0.000000	39.000000	0.000000	9.000000	29164.750000	0.000000	22.000000	9200.000000	1.000000	0.000000
50%	4.970043e+08	0.000000	45.000000	0.000000	11.000000	53529.000000	160661.000000	33.000000	14400.000000	4.000000	0.000000
75%	7.394551e+08	0.000000	51.000000	1.000000	13.000000	83231.750000	233085.000000	44.000000	20890.000000	7.000000	4647.1
max	9.999264e+08	4.000000	81.000000	5.000000	23.000000	367030.000000	885282.000000	142.000000	69740.000000	25.000000	57037.1

EDA

Visualizing Numerical Features

We need to understand the distribution of important numerical variables:

```
num_cols = ['AGE', 'YOJ', 'INCOME', 'HOME_VAL', 'BLUEBOOK', 'CAR_AGE', 'MVR_PTS', 'CLM_AMT']
```

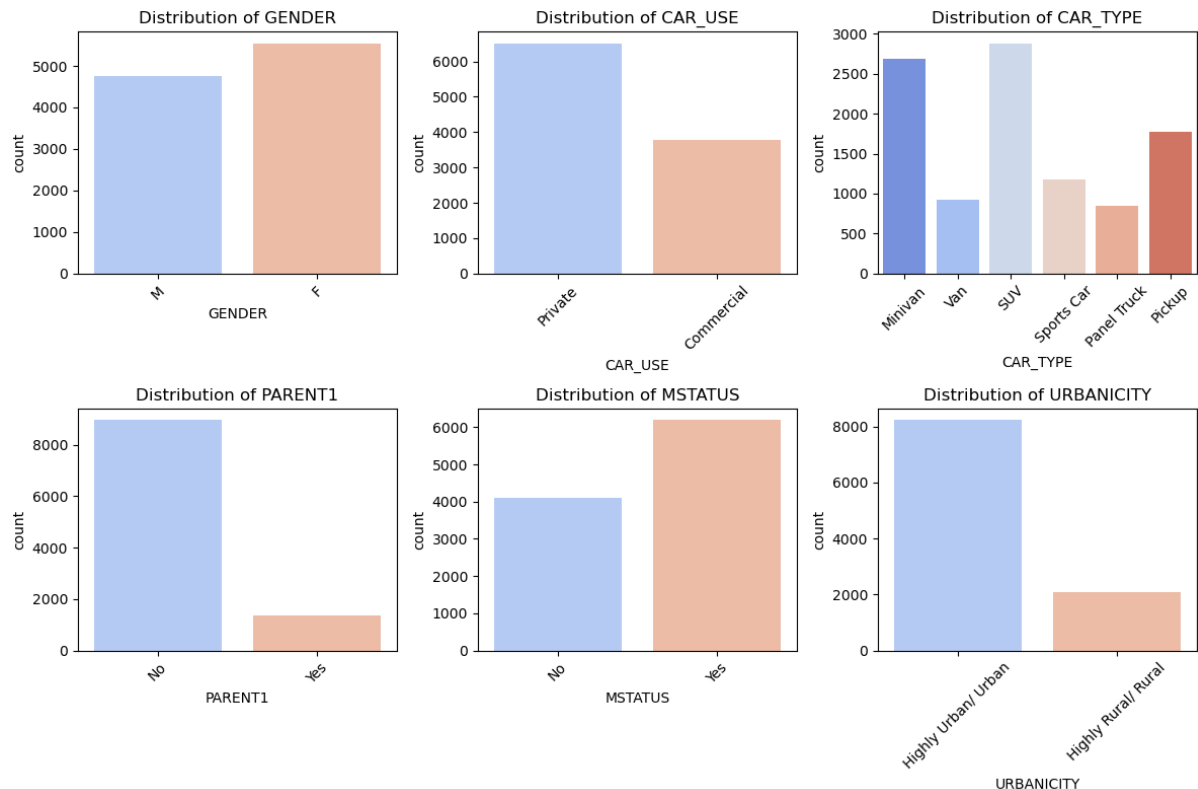


1. Distribution of age is approximately normal, centered around middle age (around 40)
2. Year of Job distribution is skewed slightly left, indicating most data points are around 11 years or less.
3. The income distribution is right-skewed, with most people having lower income of around 55k dollar, and 100k dollar for around 500 people and few with the higher income.
4. Home value this dist. is heavily right-skewed with many having lower home values around 168k
5. Bluebook this dist. is also right-skewed representing the value of the customer's car with higher value of 12k dollar.
6. Car age most cars are relatively new, with the dist skewing right
7. Motor vehicle record points: Most individuals have a low count of points.

8. Claim amount: The distribution is skewed right, indicating most claims are low, with a few large claims

Analyzing Categorical Features

cat_cols = ['GENDER', 'CAR_USE', 'CAR_TYPE', 'PARENT1', 'MSTATUS', 'URBANICITY']

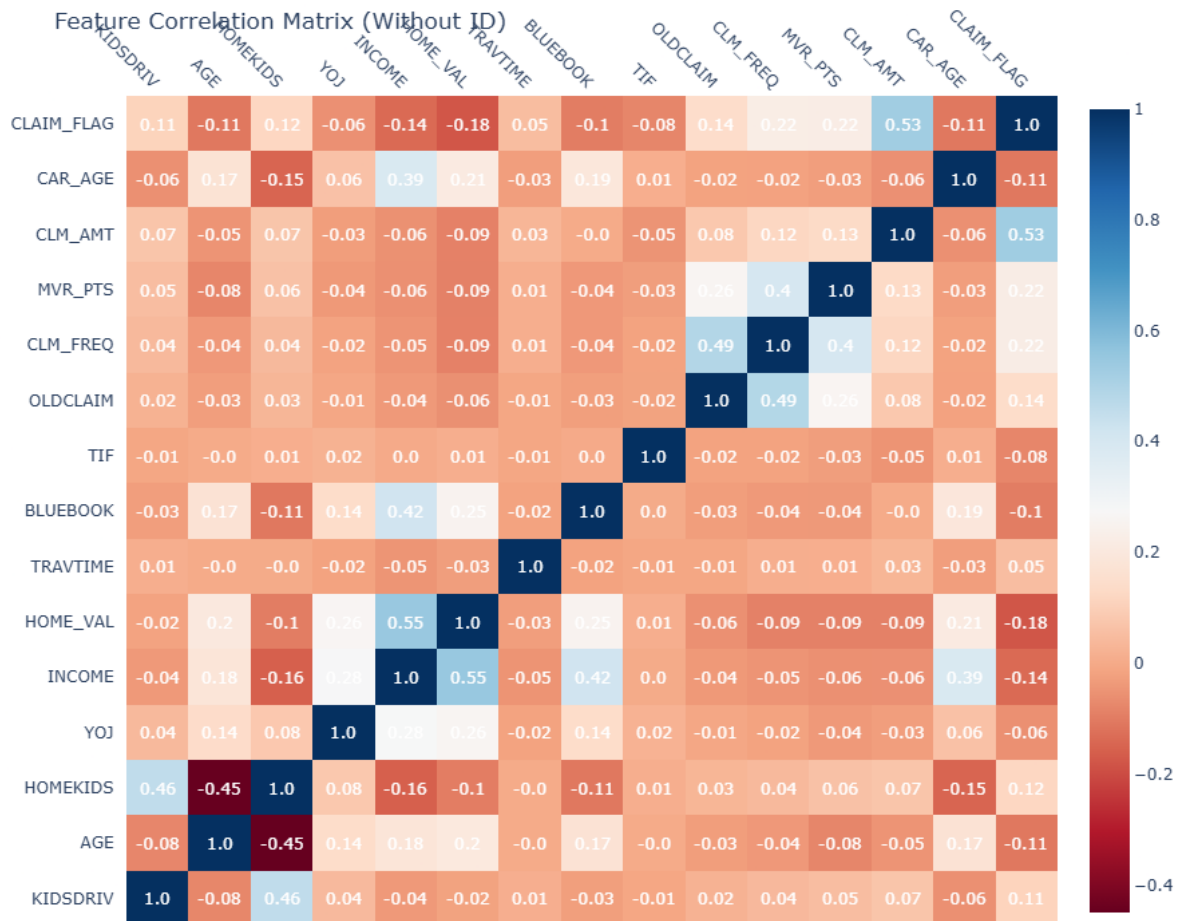


This various bar charts representing distributions of different categories.

1. In terms of gender, there are slightly more individuals identified as Female than Male.
2. Most cars are used privately rather than commercially.
3. Minivans and SUV are the most common car types, while panel trucks and Van are the least common.
4. The majority of individuals do not identify as single parents .
5. A higher number of individuals are married versus single.
6. Lastly, most individuals live in urban areas compared to rural ones.

Bivariate Analysis

Feature Correlation Matrix



Correlation values range from -1 to 1:

- Positive correlation (closer to 1): When one variable increases, the other tends to increase.
- Negative correlation (closer to -1): When one variable increases, the other tends to decrease.
- Near 0 correlation: Weak or no linear relationship between variables.

Key Observations:

1.Claim Amount (CLM_AMT) vs. Claim Flag (CLAIM_FLAG) → 0.53 (Moderate Positive Correlation) This makes sense as higher claim amounts are more likely associated with filed claims.

2.Income vs. Home Value → 0.55 (Moderate Positive Correlation)

Higher-income individuals tend to have higher home values, which aligns with real-world expectations.

3.Income vs. Bluebook (Car Value) → 0.42 (Moderate Positive Correlation)

Wealthier policyholders tend to own higher-value vehicles.

4.TIF (Time in Force) has low correlations with most variables, suggesting it may not be a strong predictor in the model.

5.HOMEKIDS vs. AGE (-0.45) → Negative Correlation

Younger individuals tend to have more children in the household, which is logical.

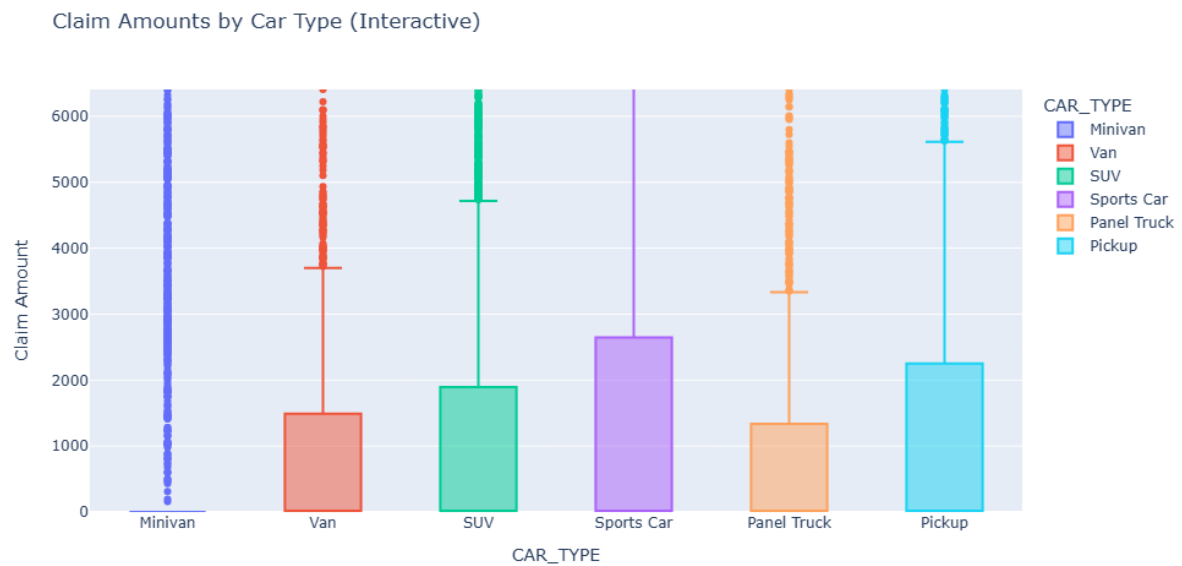
6. Car Age vs. Income (0.39)

Individuals with higher income tend to own relatively newer vehicles.

Conclusion for heatmap

- Some strong relationships exist (e.g., income and home value, claim amount and claim flag), which can be useful for predictive modeling.
- Low correlations among certain variables indicate independent features, reducing multicollinearity concerns.

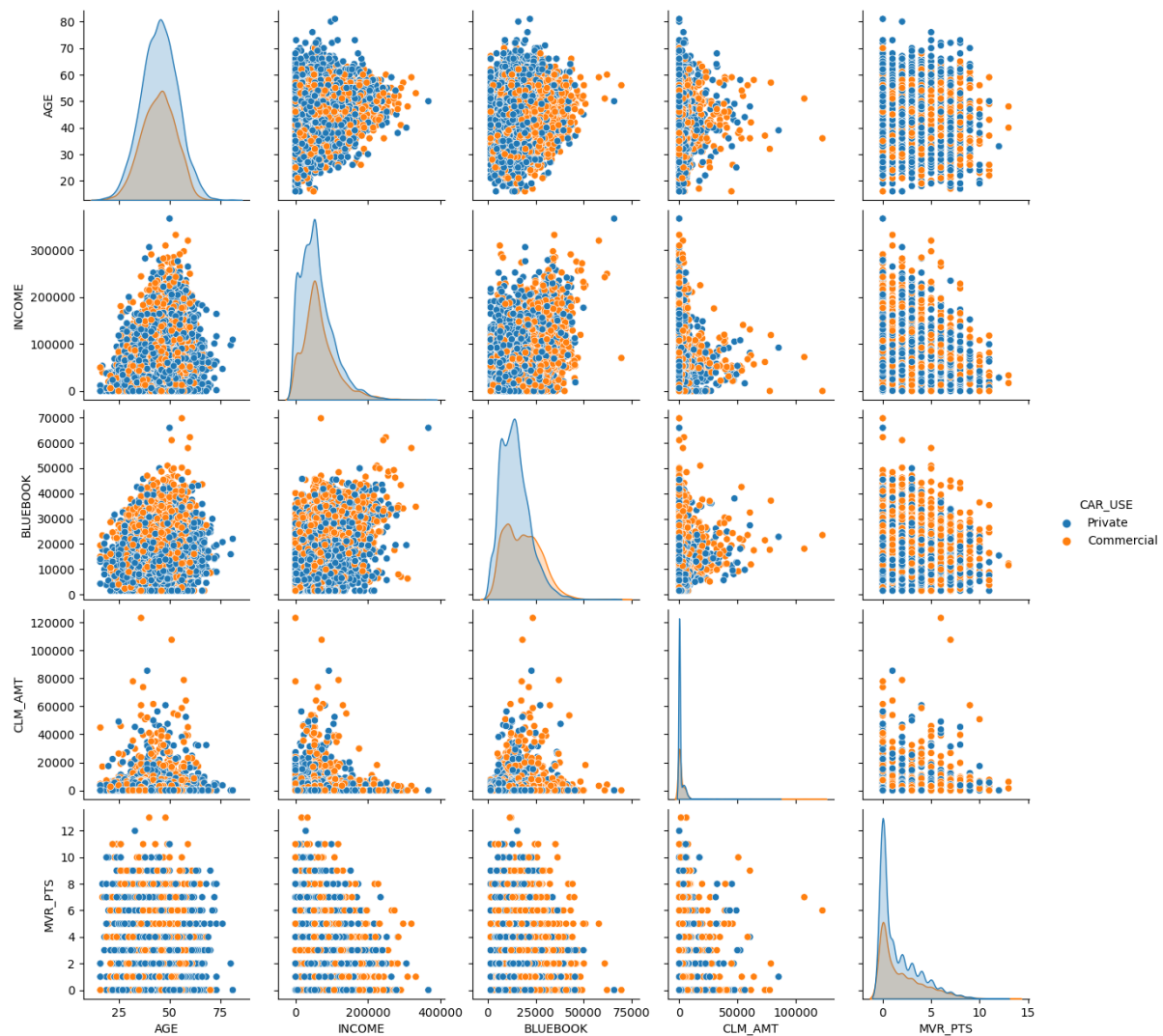
Claim amount by car type



Key Takeaways:

- Sports Cars & SUVs have the highest median claim amounts.
- Minivans and Pickups show extreme outliers, making their claims unpredictable.
- Panel Trucks have a more consistent distribution, though still showing high claims.
- Insurance companies might charge higher premiums for SUVs, Sports Cars, and Minivans due to their higher claim medians and extreme outliers.

Pairwise Scatter Plot of Numerical Features



The density plots on the diagonal in this image show that most users are in a similar age range, and income has a right-skewed distribution (a few individuals have very high incomes).

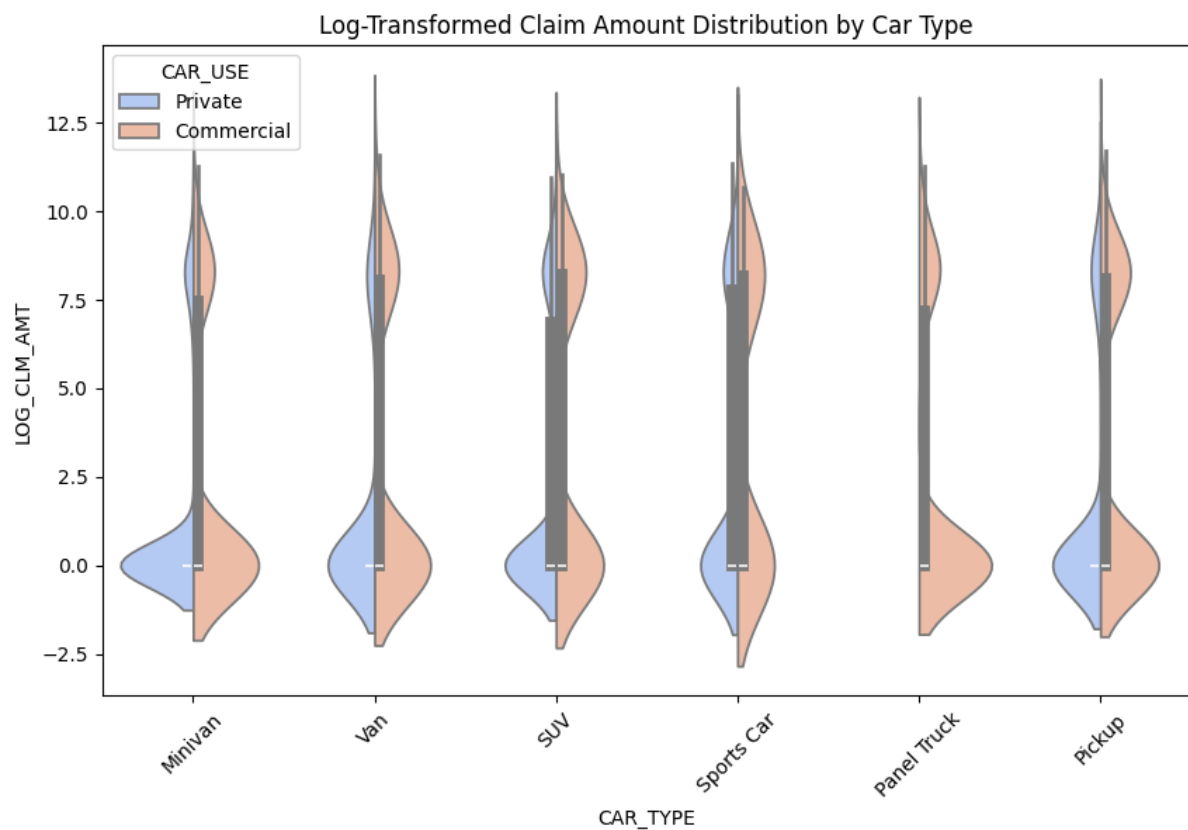
Relationship Between Variables:

- Income vs. Bluebook: There is a positive correlation, meaning higher car values tend to be associated with higher incomes.
- Claim Amount vs. Bluebook: No strong correlation, implying car value does not necessarily determine claim amounts.
- Claim Amount vs. MVR Points: Higher MVR Points (more violations) seem to be associated with slightly higher claim amounts.

Private vs. Commercial Cars:

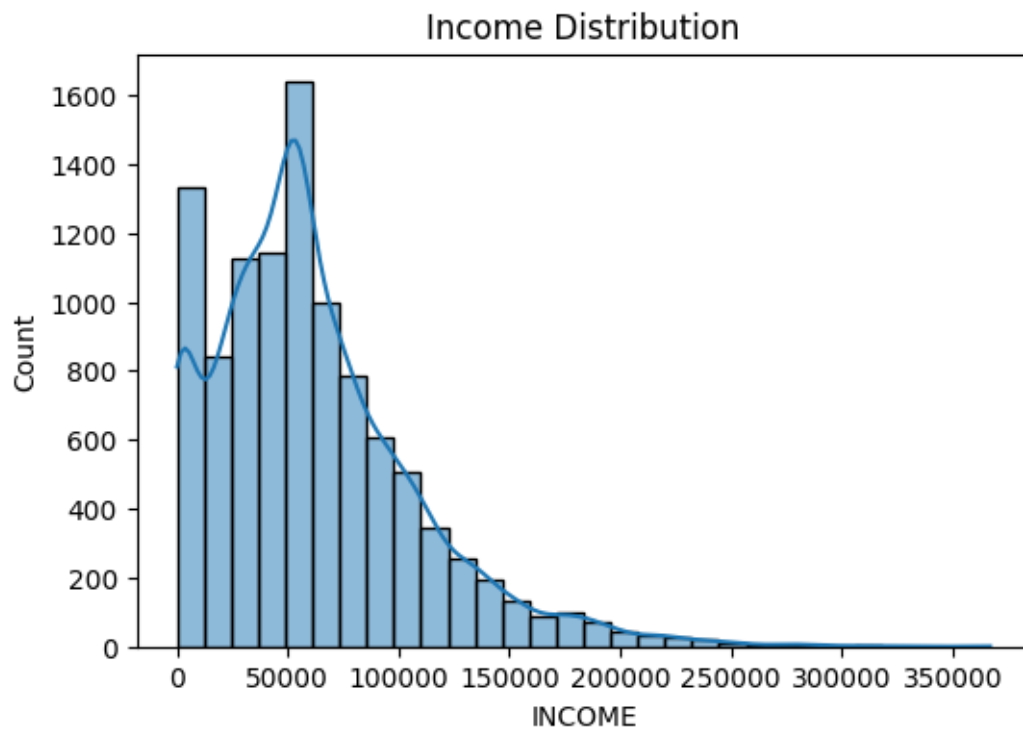
- The two categories are overlapping in most cases, but slight differences exist.
- Commercial users (orange dots) seem to be more spread across income and claim amounts.

Claim Amount dist by Car type and its usage

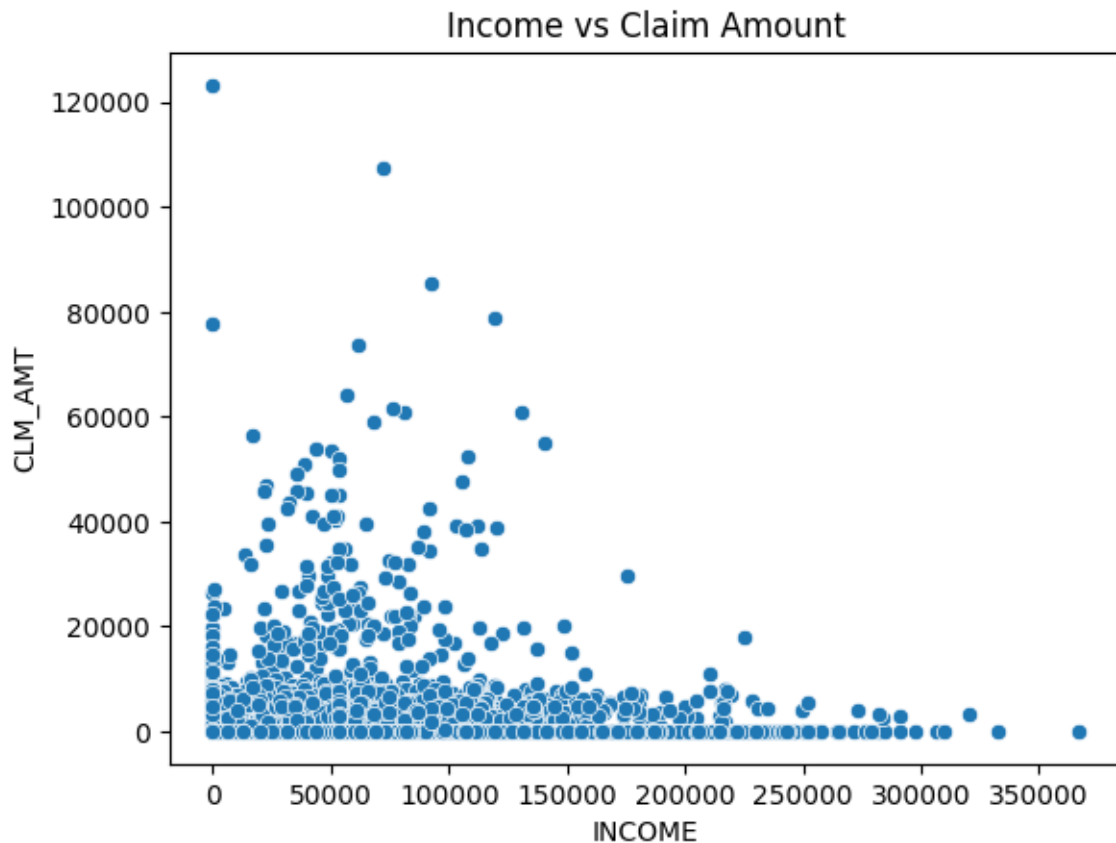


- Comparison Between Private vs. Commercial Vehicles Commercial vehicles (orange) tend to have higher claim amounts on average compared to private vehicles.
- The upper portion of the violin plots shows that commercial vehicles have a wider range of claim amounts.
- Business Implications Insurance companies should be cautious about commercial vehicles, as they have a higher risk of large claims.
- Certain vehicle types (like SUVs & Vans) have a wider spread, suggesting more unpredictable claim amounts.
- Pricing strategies for insurance premiums could be adjusted based on vehicle type and usage.

Income distribution

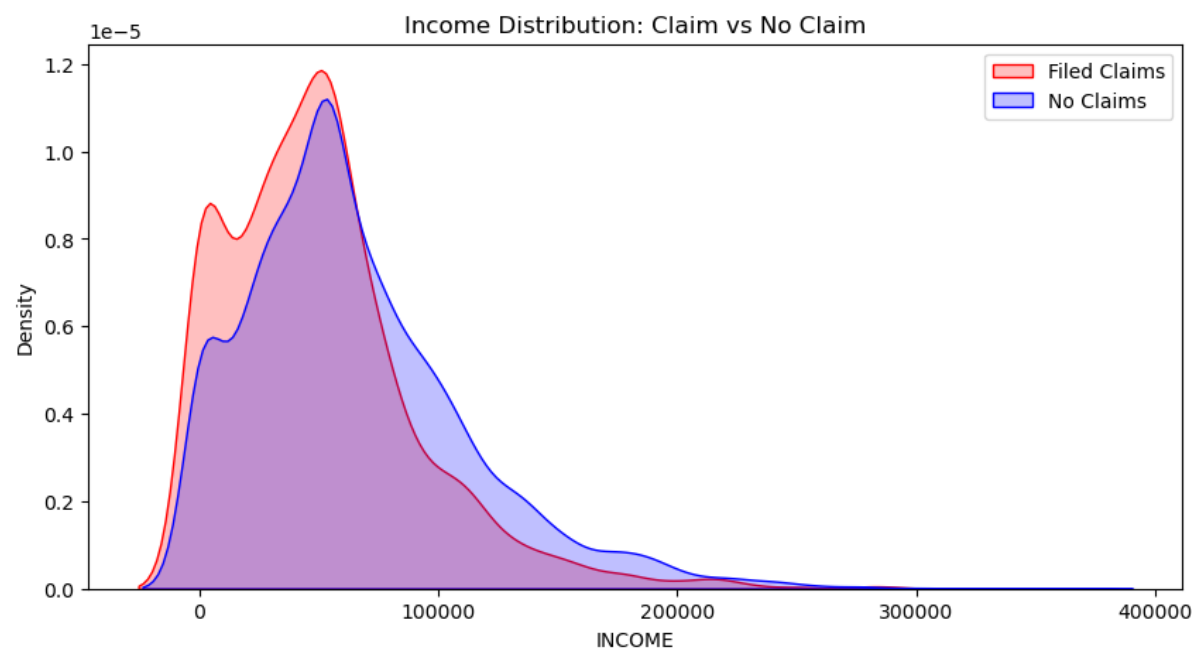


```
count      10302.000000
mean       61127.057562
std        46162.151534
min         0.000000
25%        29164.750000
50%        53529.000000
75%        83231.750000
max        367030.000000
Name: INCOME, dtype: float64
```



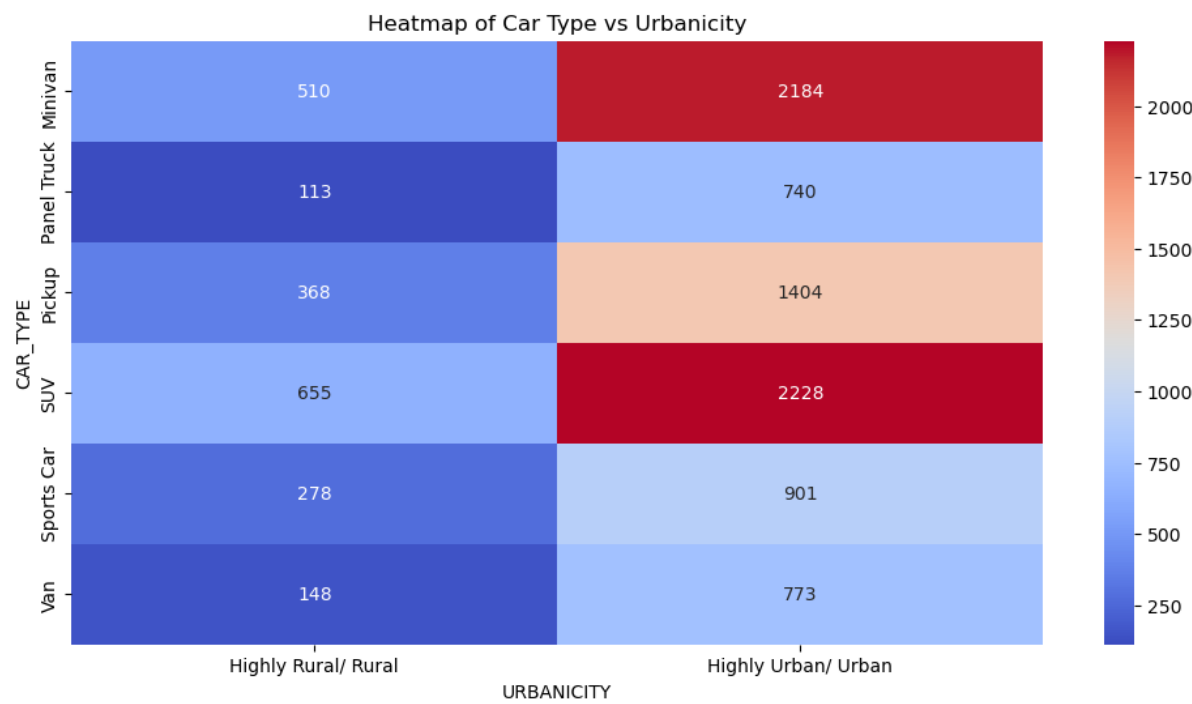
- **Lower-Income Policyholders Tend to Claim More**
From the scatter plot, we see that most high claim amounts come from people with lower incomes. This suggests an inverse relationship — as income decreases, the potential claim amount tends to increase.
- **Income is Heavily Skewed**
The income distribution plot shows that most customers earn below \$100,000, while very few have high incomes (above \$200,000). This long tail to the right indicates positive skewness, which can affect model accuracy if not addressed.
- **High Claim Risk in Low to Mid-Income Groups**
Combining both plots, we infer that low and mid-income groups may pose higher claim severity risk, making income a critical feature for predicting claim amounts and guiding pricing strategies.

Income distributions claims vs no claims



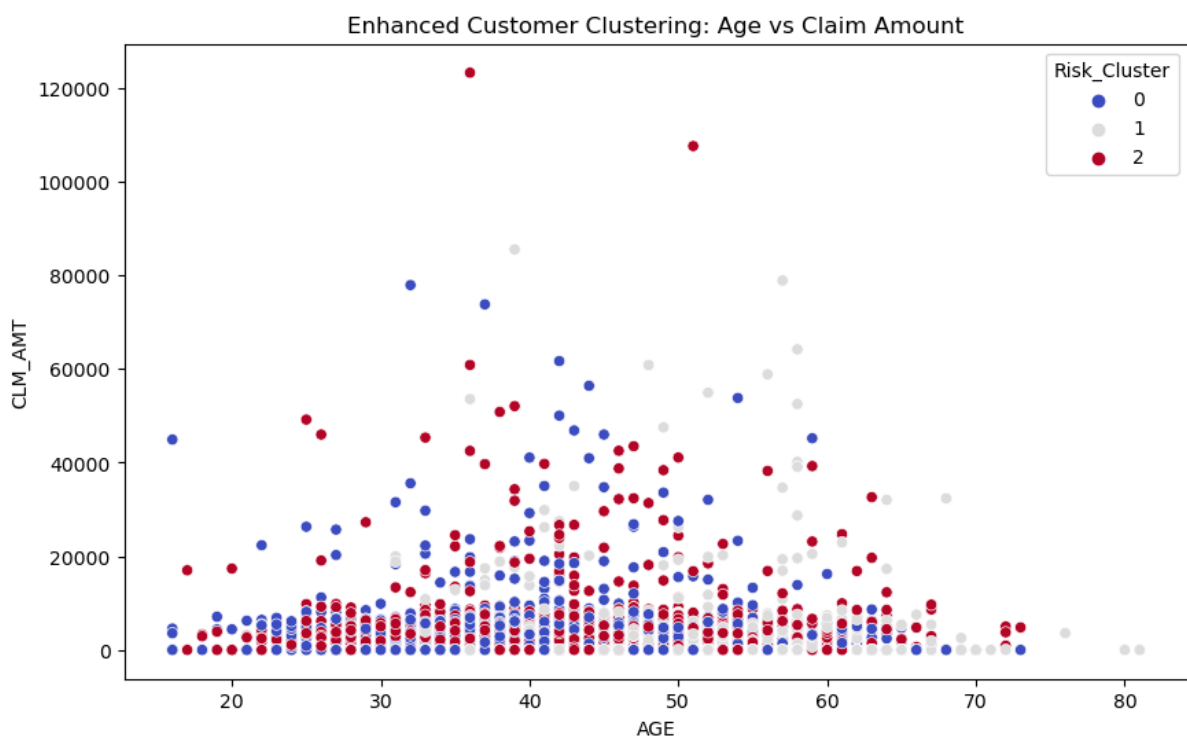
- Insights: The data suggests that lower-income individuals tend to file more claims compared to higher-income groups.
- As income increases, the likelihood of filing a claim decreases.
- This insight can be useful for insurance companies to design better risk-based pricing strategies.

Car type vs Urbanicity



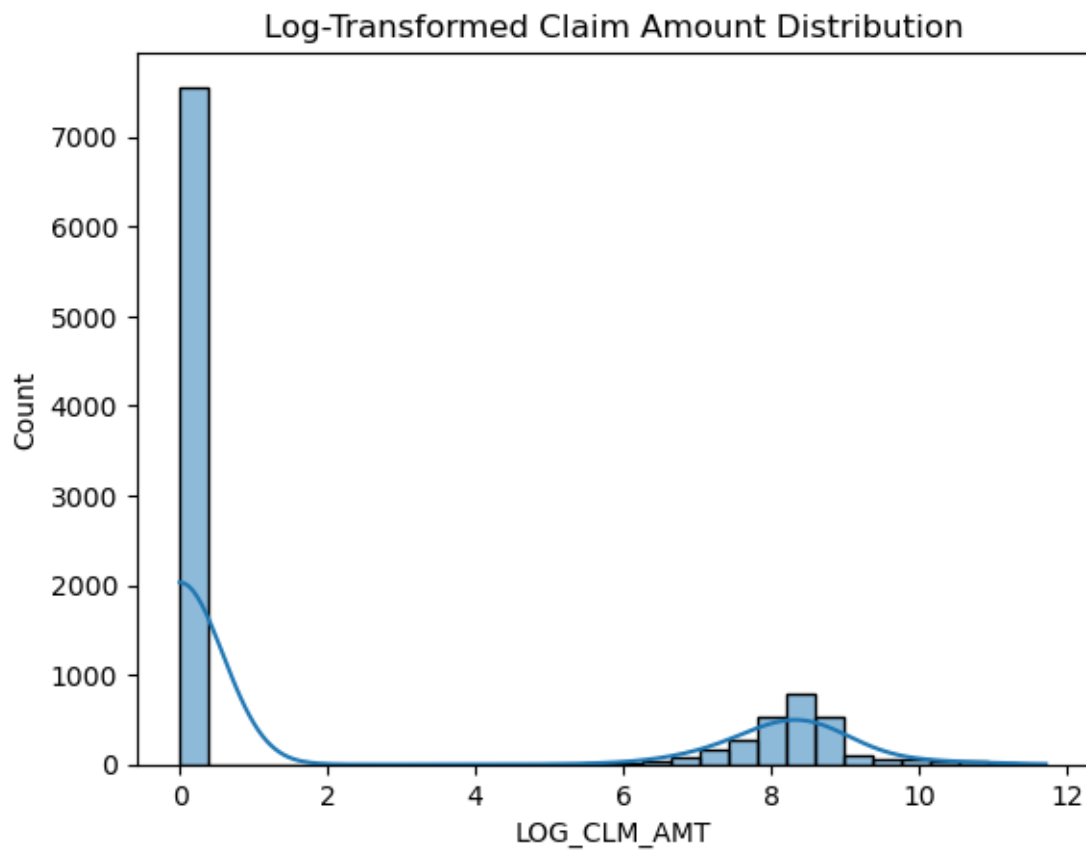
- Charge Higher Premiums for SUVs and Minivans in Urban Areas
- SUVs (2228) and Minivans (2184) are the most common car types in urban areas.
- Urban areas have higher accident rates, theft risks, and repair costs due to traffic congestion and population density.
- Action: Increase premium rates for these vehicles in urban locations to account for higher risk exposure.
- Rural customers have fewer vehicle choices (lower counts of Panel Trucks, Vans, and Sports Cars).
- Action: Offer pay-as-you-drive or low-mileage discounts for rural customers, as they drive less and pose lower risk.

Risk and Claim Trends



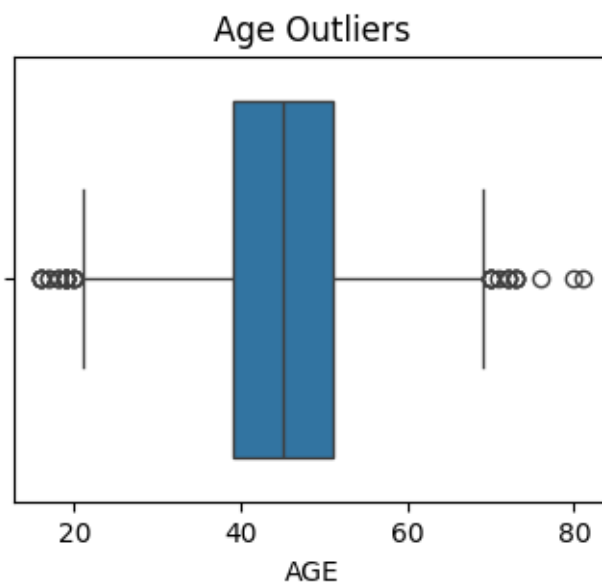
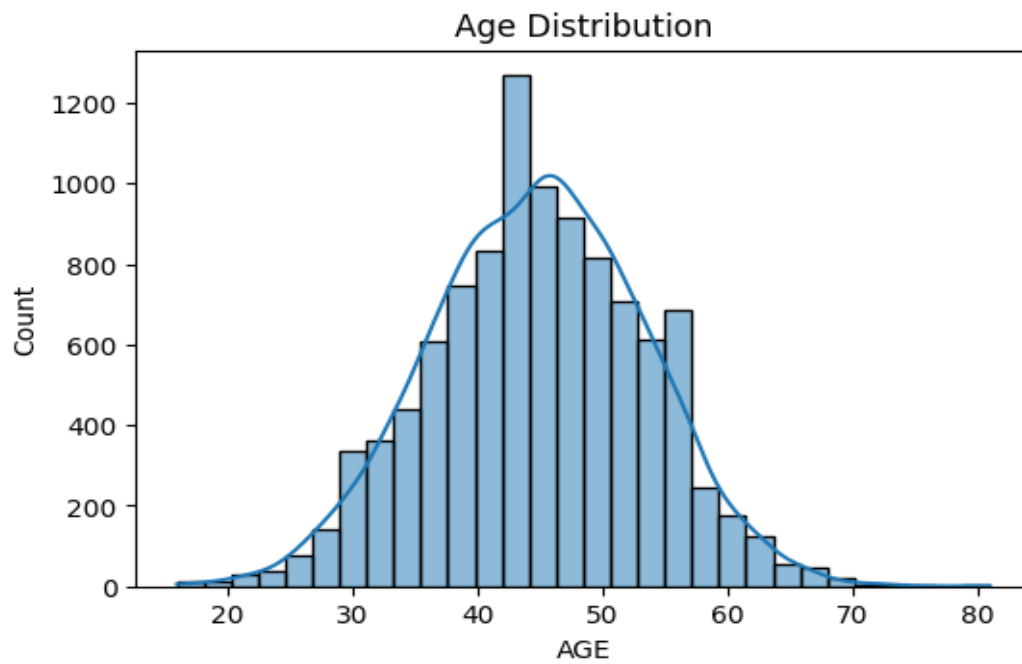
- Middle-aged customers (30-50 years old) have the highest claim amounts and the highest number of high-risk customers.
- Younger customers (under 30) generally have lower claims and are mostly low-risk.
- Older customers (50+) show moderate claim amounts, with a mix of medium and high-risk individuals.

Log Tranformed Claim Amount Distribution



- The large number of zero-claim customers suggests that many policyholders may be low-risk, opening opportunities for targeted discounts or lower premiums.
- The right-skewed claim distribution indicates that a small percentage of customers file high claims, meaning insurers should focus on risk assessment and fraud detection for high-claim segments.

Age



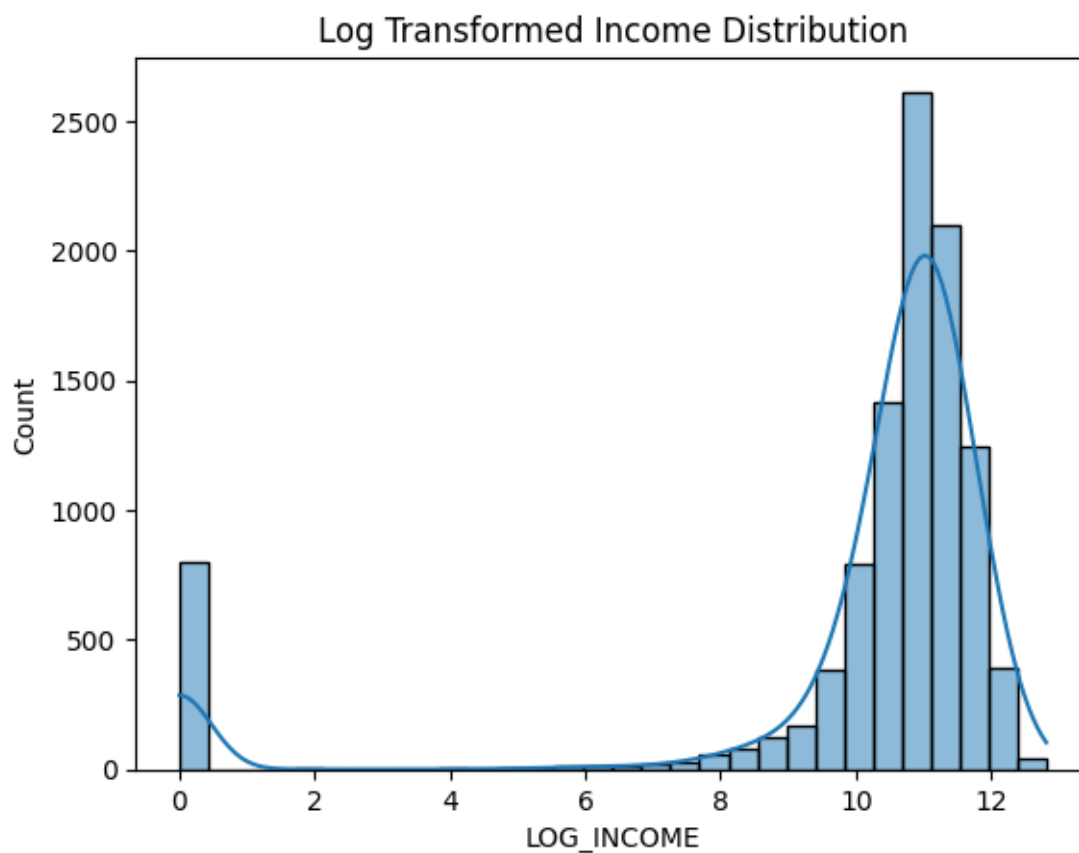
```
count    10302.000000
mean      44.837507
std       8.603521
min       16.000000
25%       39.000000
50%       45.000000
75%       51.000000
max       81.000000
Name: AGE, dtype: float64
```


Senior 6772
Middle 3083
Elder 327
Young 120

The age distribution is approximately normal, with a peak around 45 years.
Most individuals fall between 39 and 51 years (IQR), while the mean age is 44.84 years.
The majority of customers belong to the "Senior" (6772) and "Middle" (3083) age groups.

Outliers are observed at both extremes (below 20 and above 70 years).
These could represent exceptional cases, such as early or late insurance claimants.

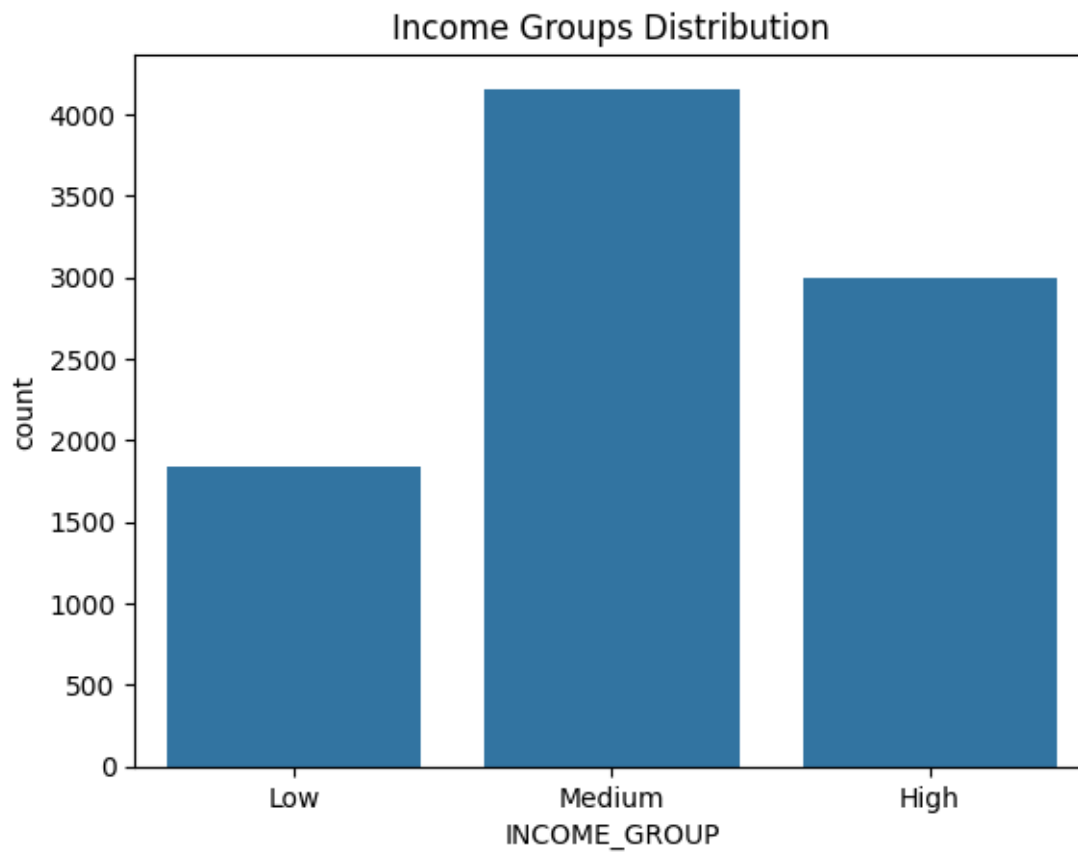
Log Transformed Income Distribution



Mode (Peak) of log-income:

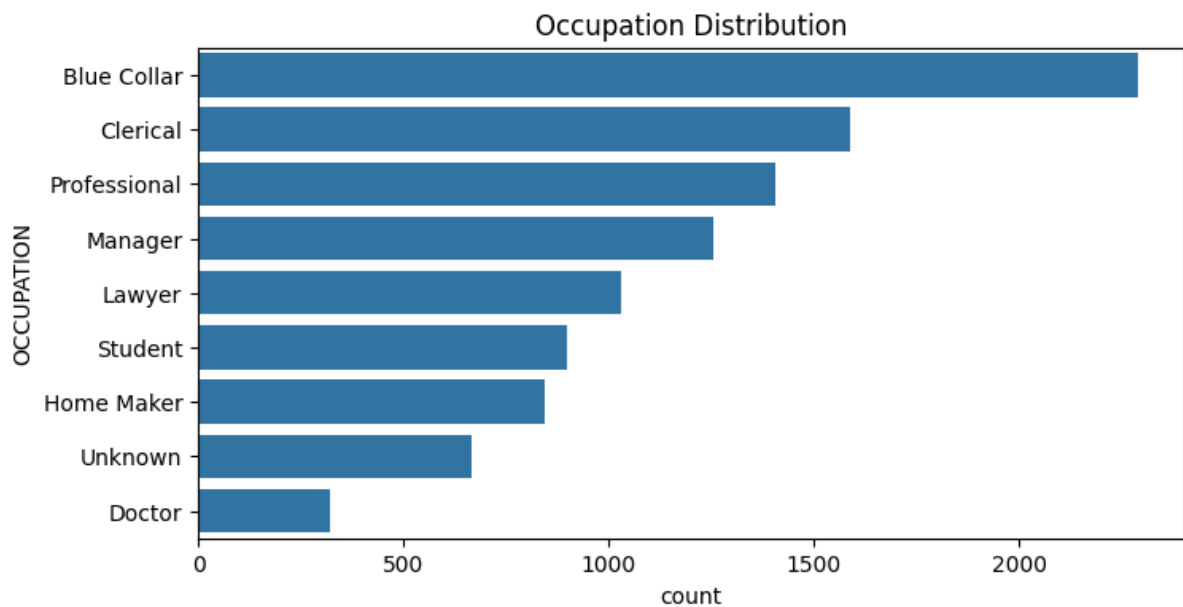
- The peak appears around log-income values between 10 and 11, suggesting that the most common income levels, after log transformation, fall in this range.
- So most individuals are likely earning 22K to 60K annually.
- Zero or Very Low Incomes: A sharp spike at log-income = 0 suggests a significant number of individuals with zero or very low reported income.

Income Group Distribution



- Most People Have a Medium Income: The largest group (4,157 people) falls in the medium-income range, meaning their earnings are around the average.
- High-Income Group is Significant: A good number (2,995 people) earn a high income, but they are fewer than those in the medium category.
- Fewer People Have Low Income: Only 1,844 people fall into the low-income category, making them the smallest group.

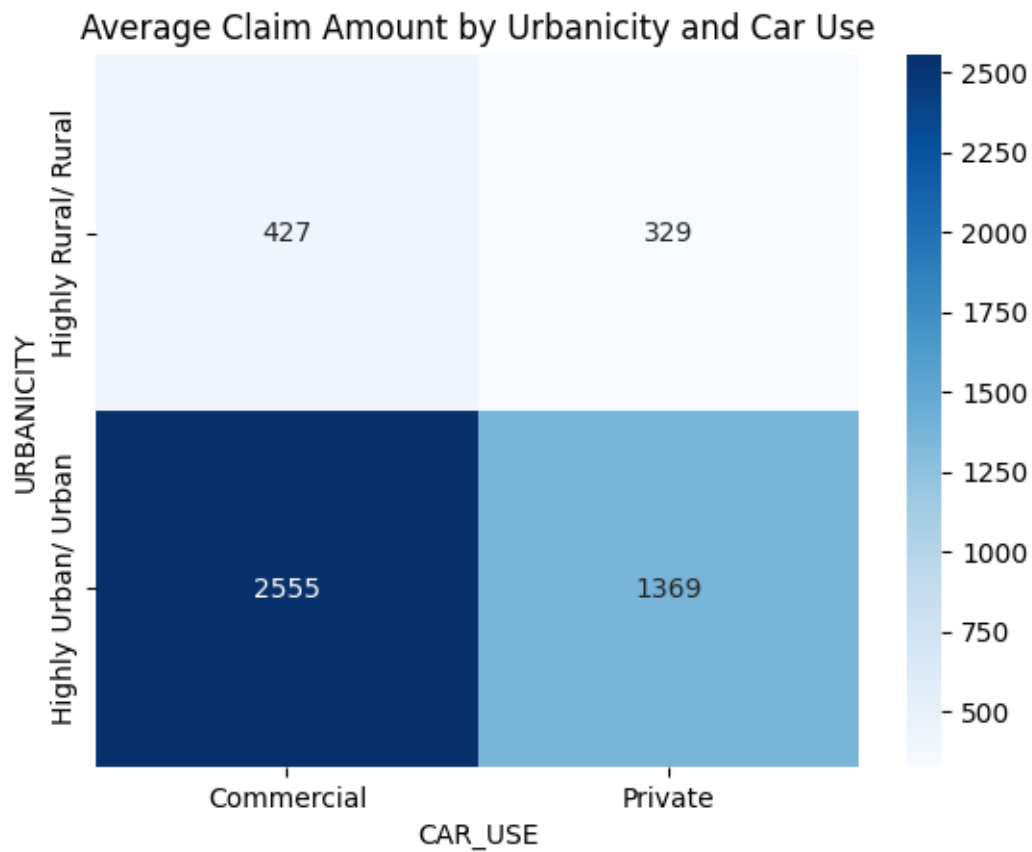
Occupation Distribution



Blue Collar	2288
Clerical	1590
Professional	1408
Manager	1257
Lawyer	1031
Student	899
Home Maker	843
Unknown	665
Doctor	321

- Most Common Occupation: Blue-collar jobs (2,288 people) dominate the dataset.
- Office and Skilled Jobs: Clerical (1,590), professional (1,408), and managerial (1,257) roles are significant.
- Specialized Professions: Lawyers (1,031) and doctors (321) represent high-skilled fields.
- Students and Home Makers: A notable portion includes students (899) and home makers (843).
- Unknown Category: 665 individuals have unspecified occupations, possibly retirees or unemployed.

Urbanicity and Car Use



The combination of urban location and commercial use leads to the highest claim severity, while rural private cars pose the least financial risk in terms of average claim amounts. This kind of segmentation is valuable for underwriting, pricing, and risk assessment.

Claim Amount by Occupation

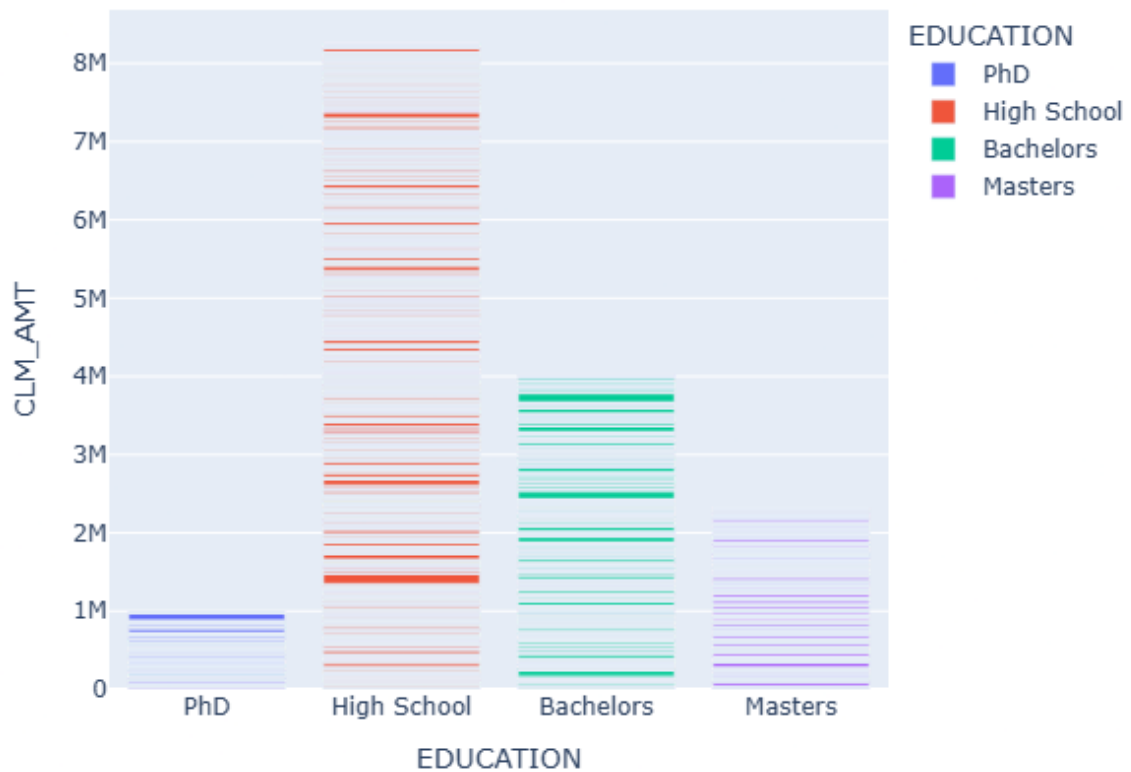
Claim Amount by Occupation



- Top Occupation – Blue Collar Blue Collar workers have the highest total claim amount at 4,752,674. This indicates they are either involved in more accidents or more expensive ones, possibly due to the nature of their jobs (e.g., physical labor, higher road exposure).
- Other High Claim Categories Clerical (2.57M), Professional (2.1M), and Student (1.77M) occupations also show significant total claim amounts. Clerical roles might involve regular commuting; students may be inexperienced drivers.
- Lowest Claim Amount – Doctor Doctors have the lowest claim total, suggesting fewer claims or lower amounts per claim, possibly due to lower driving frequency or more cautious driving behavior.

Claim amount vs Education

Average Claim Amount by Education Level

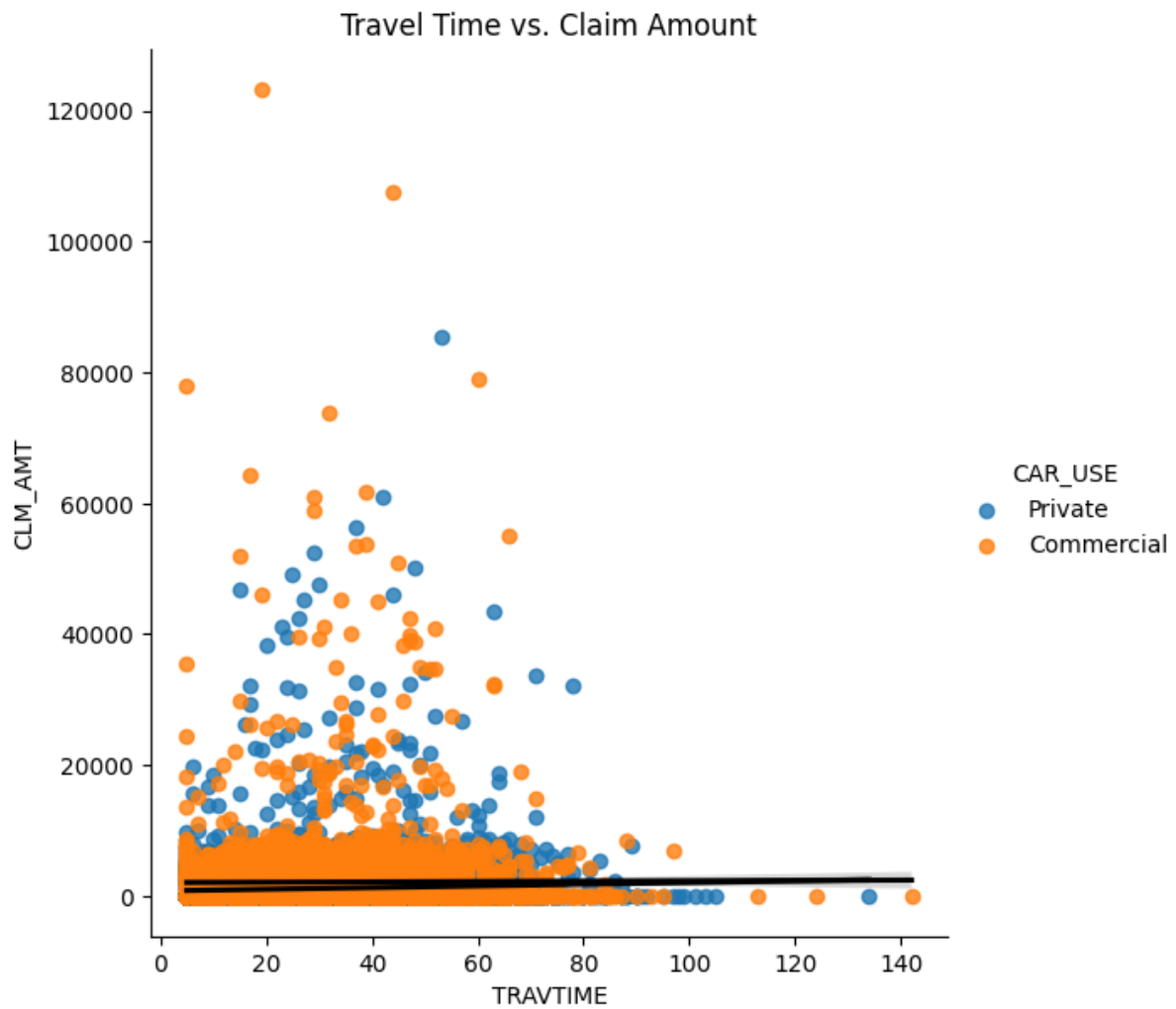


- Inverse Relationship Between Education and Claim Amounts: Generally, as education level increases, claim amounts tend to decrease.

Potential Business Insight for Insurers:

- Individuals with only a high school education might represent a higher insurance risk segment compared to those with advanced degrees.

Claim amount vs Time Travel



- Commercial use may be a risk factor for higher claim amounts.
- Travel time is not a strong predictor of claim amount on its own.
- Most incidents (and claims) occur during shorter trips, which may be due to higher trip frequency or urban driving conditions.
- Outliers (very high claims) are more often associated with commercial use, which could inform premium pricing models.

Feature Engineering

Feature engineering plays a critical role in improving the predictive performance of machine learning models. In this study, we applied various feature transformations, new feature creations, and selection techniques to enhance data quality and reduce redundancy. The key steps taken are detailed below.

GENDER and URBANICITY

```
df['GENDER'] = df['GENDER'].map({'M': 0, 'F': 1})
df['URBANICITY'] = df['URBANICITY'].map({'Highly Rural/ Rural': 0, 'Highly Urban/ Urban': 1})
```

Convert categorical variables (GENDER and URBANICITY) into numerical values for machine learning compatibility.

Transformation Applied:

- GENDER: Mapped 'M' → 0 and 'F' → 1 (Binary Encoding).
- URBANICITY: Mapped 'Highly Rural/ Rural' → 0 and 'Highly Urban/ Urban' → 1

Binary encoding for cats

```
# Binary encoding for categorical variables
binary_cols = ['PARENT1', 'MSTATUS', 'REVOKED']
for col in binary_cols:
    df[col] = df[col].map({'Yes': 1, 'No': 0})
```

PARENT1(Is the policyholder a parent?) : YES>1, NO>0

MSTATUS (Marital Status of the policyholder) yes>1 indicating married, NO>0 indicating single

REVOKED (Has the policyholder's license been revoked?) YES >1, NO > 0

Ordinal Encoding for Education

```
df['EDUCATION'] = df['EDUCATION'].replace(education_mapping)

# Fill NaN values with 'Unknown'
df['EDUCATION'].fillna('Unknown', inplace=True)

# Define the correct order (matching the standardized categories)
education_order = [['Unknown', 'Below High School', 'High School', 'Bachelor', 'Master', 'PhD']]
|

# Initialize OrdinalEncoder
ordinal_encoder = OrdinalEncoder(categories=education_order)

# Apply encoding
df[['EDUCATION']] = ordinal_encoder.fit_transform(df[['EDUCATION']])

# Convert EDUCATION column to integers for better readability
df['EDUCATION'] = df['EDUCATION'].astype(int)

# Display the first few rows
print(df[['EDUCATION']].head())
```


Any missing (NaN) values in the EDUCATION column replaced with 'Unknown' .
ranking (High School < Bachelor < Master < PhD), so **Ordinal Encoding** was applied.

Education level Encoded value

Unknown :0
Below high school :1
High School :2
Bachelor: 3
Masters:4
phd :5

One-hot encoding for car type

```
df = pd.get_dummies(df, columns=['CAR_TYPE'], prefix='CT', drop_first=True)
```

Creates binary columns for each unique car type.
Drops the first category (drop_first=True) to avoid the dummy variable trap
One-hot encoding is chosen since car types are categorical but non-ordinal (no ranking relationship).
Dropping the first category prevents redundant information, ensuring model efficiency.

One-hot encoding for Occupation type

```
df = pd.get_dummies(df, columns=['OCCUPATION'], prefix='OCC', drop_first=True)
```

Same as car type for Occupation.

Claim fre, Risk Score, Claim per car age, Claim to Income

```
# Claim Frequency per Time
df['CLAIM_RATE'] = df['CLM_FREQ'] / (df['TIF'] + 1)

# Risk Score based on violations
df['RISK_SCORE'] = df['MVR_PTS'] * df['CLM_FREQ']

# Claim per Car Age
df['CLAIM_AGE_RATIO'] = df['OLDCLAIM'] / (df['CAR_AGE'] + 1)

# Claim to Income Ratio
df['CLAIM_TO_INCOME_RATIO'] = df['CLM_AMT'] / (df['INCOME'] + 1)
```

1.Claim Rate Calculation (CLAIM_RATE):
CLM_FREQ: Number of claims made.
TIF: Time in force (policy duration).

Why?

1. Avoids division by zero by adding 1 to TIF.
2. Measures the frequency of claims relative to the policy's age.
3. Helps identify customers who make frequent claims in a short time.

2.Risk Score (RISK_SCORE):

MVR_PTS: Motor Vehicle Report points (higher points indicate more violations).

CLM_FREQ: Number of claims made.

Why?

1. Directly correlates past violations with claim frequency.
2. Helps identify high-risk policyholders.

3.Claim Per Car Age (CLAIM_AGE_RATIO)

OLDCLAIM: Total past claim amount.

CAR_AGE: Age of the insured vehicle.

Why?

1. Avoids division by zero by adding 1 to CAR_AGE.
2. Highlights whether older cars have higher claim costs.
3. Helps in risk assessment based on vehicle age.

4.Claim to Income Ratio (CLAIM_TO_INCOME_RATIO)

CLM_AMT: Total claim amount.

INCOME: Policyholder's income.

Why?

1. Identifies customers whose claims are high relative to their income.
2. Helps detect fraudulent or high-risk claims.

Winsorization for Claim amount

```
from scipy.stats.mstats import winsorize

# Winsorize claim amount
df['CLM_AMT_WINSORIZED'] = winsorize(df['CLM_AMT'], limits=[0.05, 0.05])
```

What is Winsorization?

Winsorization is a statistical technique used to limit extreme values by capping them at a certain percentile. Instead of removing outliers, it replaces extreme values with the nearest percentile boundary values.

limits=[0.05, 0.05]:

- Bottom 5% of values are replaced by the value at the 5th percentile.
- Top 5% of values are replaced by the value at the 95th percentile.

Impact on the Dataset

- CLM_AMT_WINSORIZED is now a more stable version of CLM_AMT.
- Extreme claims that could have biased risk models are now adjusted.
- Helps in better regression modeling and fraud detection.

Log transformation

```
df['LOG_CLAIM_RATE'] = np.log1p(df['CLAIM_RATE'])
df['LOG_INCOME'] = np.log1p(df['INCOME'])
df['LOG_BLUEBOOK'] = np.log1p(df['BLUEBOOK'])
```

Why?

Log transformation is used to reduce skewness and normalize right-skewed data, making it more suitable for modeling.

How?

- `np.log1p(x)` applies $\log(1 + x)$ to handle zero values safely.
- It compresses large values while spreading smaller ones, improving model performance.

Effect on Features:

LOG_CLAIM_RATE: Stabilizes claim rate distribution.

LOG_INCOME: Reduces income disparity effects.

LOG_BLUEBOOK: Normalizes vehicle price (BLUEBOOK).

Makes data more linear and enhances predictive modeling efficiency.

Old Car Indicator

```
# Old Car Indicator
df['OLD_CAR'] = (df['CAR_AGE'] > 10).astype(int)
```

Why?

Creating an "OLD_CAR" indicator helps categorize vehicles into old (1) vs. new (0), making it easier for models to identify patterns related to older cars.

How?

- Condition: `df['CAR_AGE'] > 10` checks if a car is older than 10 years.
- `astype(int)`: Converts True/False to 1/0 for better model compatibility.

Helps assess risk based on car age.

Simplifies interpretation for decision trees and regression models.

Driver Experience Calculation

```
df['BIRTH'] = pd.to_datetime(df['BIRTH'], errors='coerce')
```

```
# Driver experience calculation
current_year = pd.Timestamp.now().year
df['DRIVING_EXPERIENCE'] = current_year - df['BIRTH'].dt.year - 16

df['GENERATION'] = pd.cut(df['BIRTH'].dt.year,
                          bins=[1925, 1945, 1965, 1980, 1995, 2010],
                          labels=['Silent', 'Baby Boomer', 'Gen X', 'Millennial', 'Gen Z'])
```

Why?

"DRIVING_EXPERIENCE" calculates the number of years a driver has been eligible to drive (assuming a minimum driving age of 16 years).

"GENERATION" categorizes drivers based on birth year into generational groups, which may help identify trends in driving behavior.

How?

1. Convert BIRTH to datetime → Ensures proper date calculations.
2. Calculate driving experience → $\text{current_year} - \text{birth_year} - 16$ (adjusting for minimum driving age).
3. Categorize generation using `pd.cut()` → Assigns birth years into Silent, Baby Boomer, Gen X, Millennial, and Gen Z.

Effect :

Helps in risk assessment (e.g., younger drivers may have more claims).

Useful for customer segmentation and policy pricing.

Job stability index

```
# Job stability index
df['JOB_STABILITY'] = df['YOJ'] / (df['AGE'] - 18 + 1)

# Career interruption flag
df['CAREER_GAP'] = ((df['AGE'] - 18) > df['YOJ']).astype(int)
```

"JOB_STABILITY" estimates how long an individual has been in their job relative to their working-age years.

"CAREER_GAP" identifies whether a person had a career break before starting their current job.

Job Stability Index:

$\text{YOJ} / (\text{AGE} - 18 + 1)$, where YOJ (Years on Job) is divided by total working years (assuming work starts at 18).

+1 avoids division by zero for those aged 18.

Career Gap Flag:

If "Years on Job" (YOJ) is less than total working years ($\text{AGE} - 18$), it indicates a career gap.

Converted to binary (1 = gap, 0 = no gap) for easy interpretation.

Effect

Helps assess employment stability and career interruptions, useful for risk profiling and financial behavior prediction.

Family responsibility score

```
# Family responsibility score
df['FAMILY_RISK'] = (df['PARENT1'] +
                    df['HOMEKIDS'] * 0.5 +
                    df['KIDSDRIV'] * 0.7)

# Teen driver risk
df['TEEN_DRIVER_RISK'] = df['KIDSDRIV'] * df['MVR_PTS']
```

"FAMILY_RISK" quantifies financial and responsibility burden based on family structure.
"TEEN_DRIVER_RISK" evaluates risk associated with teenage drivers, considering past driving violations.

Family Responsibility Score (FAMILY_RISK)

PARENT1 (Single parent) contributes 1 to risk.

HOMEKIDS (Children at home) adds 0.5 per child.

KIDSDRIV (Kids who drive) adds 0.7 per teen driver (higher weight due to driving risk).

Teen Driver Risk (TEEN_DRIVER_RISK)

Multiplying teen drivers (KIDSDRIV) by past violations (MVR_PTS) increases risk for families with inexperienced, high-risk drivers.

Helps assess financial burden and driving risk, valuable for insurance pricing and fraud detection.

Urban wealth indicator

```
# Urban wealth indicator
df['URBAN_WEALTH'] = df['URBANICITY'] * df['INCOME_GROUP']

df['RISKY_VEHICLE'] = ((df[['CT_SUV', 'CT_Sports Car']].sum(axis=1) > 0) &
                       (df['BLUEBOOK'] > df['BLUEBOOK'].median()))).astype(int)
```

"URBAN_WEALTH" identifies income levels in urban vs. rural areas, helping assess financial stability.

"RISKY_VEHICLE" flags expensive SUVs and sports cars, which are linked to higher accident and theft risks.

Urban Wealth Indicator (URBAN_WEALTH)

Multiplies urban/rural category (URBANICITY) with income group (INCOME_GROUP) to create a financial status metric.

Risky Vehicle (RISKY_VEHICLE)

Checks if a driver owns either an SUV or Sports Car (CT_SUV or CT_Sports Car).

If the vehicle's market value (BLUEBOOK) is above the median, it is considered risky.

Effect :

Helps in insurance risk profiling, detecting high-risk drivers, and refining premium pricing models.

Claim acceleration rate

```
# Claim acceleration rate
df['CLAIM_ACCELERATION'] = df['CLM_FREQ'] / df['TIF']

# Policy Lifetime value
df['POLICY_LTV'] = df['TIF'] * df['INCOME'] / 1000
```

"CLAIM_ACCELERATION" helps measure how frequently a customer files claims relative to their policy duration.

"POLICY_LTV" estimates the long-term financial value of a policyholder based on tenure and income.

Claim Acceleration Rate (CLAIM_ACCELERATION)

Formula: $\text{CLM_FREQ} / \text{TIF}$

Indicates how often claims occur per year of policy tenure (TIF).

Higher values suggest high-risk policyholders who claim frequently.

Policy Lifetime Value (POLICY_LTV)

Formula: $(\text{TIF} * \text{INCOME}) / 1000$

Measures a customer's lifetime worth in the insurance system.

Higher values suggest financially valuable, long-term customers.

Effect:

Helps **insurers prioritize high-value customers** while identifying **frequent claim filers** for risk assessment.

Yeo-Johnson transformation

```
from sklearn.preprocessing import PowerTransformer

# Apply Yeo-Johnson transformation
pt = PowerTransformer()
transform_features = ['INCOME', 'HOME_VAL', 'BLUEBOOK']
df[transform_features] = pt.fit_transform(df[transform_features])
```

The Yeo-Johnson transformation helps normalize skewed data and make distributions more Gaussian, improving model performance.

Works for both positive and negative values (unlike log transformation).

- Initialize the PowerTransformer (`pt = PowerTransformer()`).
- Apply transformation on selected numerical features (INCOME, HOME_VAL, BLUEBOOK).
- Fit and transform the data (`df[transform_features] = pt.fit_transform(df[transform_features])`).

Effect:

Reduces **skewness**, improves **feature scaling**, and enhances **model performance** by stabilizing variance.

Claim severity ration

```
# Claim severity ratio
df['SEVERITY_RATIO'] = df['CLM_AMT_WINSORIZED'] / df['BLUEBOOK']

# Risk-adjusted premium
df['RISK_ADJ_PREMIUM'] = (df['MVR_PTS'] * 0.2 +
                           df['CLM_FREQ'] * 0.3 +
                           df['CAR_AGE'] * 0.1)
```

Why?

- SEVERITY_RATIO: Measures claim severity relative to the vehicle's market value (BLUEBOOK).
- RISK_ADJ_PREMIUM: A weighted formula to estimate a risk-based insurance premium, considering violations (MVR_PTS), claim frequency (CLM_FREQ), and car age (CAR_AGE).

How?

- SEVERITY_RATIO = Claim Amount (Winsorized) ÷ Car's Market Value (BLUEBOOK) → Helps insurers assess the severity of claims.
- RISK_ADJ_PREMIUM = Weighted sum of risk factors to estimate the premium adjustment based on past driving behavior.

Effect on Analysis:

- Helps in risk assessment, fraud detection, and insurance pricing strategies.

Urban density multiplier

```
# Urban density multiplier
df['URBAN_RISK_MULTIPLIER'] = np.where(
    df['URBANICITY'] == 1,
    1.25 * df['TRAVTIME'],
    0.9 * df['TRAVTIME']
)

# Rural claim penalty
df['RURAL_PENALTY'] = ((df['URBANICITY'] == 0) &
                       (df['CAR_USE'] == 'Commercial')).astype(int) * 1.15
```

URBAN_RISK_MULTIPLIER: City driving is riskier due to traffic and longer travel times.

RURAL_PENALTY: Rural commercial vehicles might have higher claim costs due to limited services.

URBAN_RISK_MULTIPLIER:

If Urban (1) → Increase travel time risk by 25%.

If Rural (0) → Reduce travel time risk by 10%.

RURAL_PENALTY:

If Rural & Commercial Vehicle → Apply extra risk penalty (1.15 factor).

Effect :

Helps set fair insurance prices based on where and how the car is used.

Features Selection

```
final_features = [
    'AGE', 'GENDER', 'EDUCATION', 'PARENT1', 'MSTATUS', 'INCOME',
    'HOME_VAL', 'BLUEBOOK', 'LOG_INCOME', 'LOG_BLUEBOOK', 'MVR_PTS',
    'REVOKED', 'TRAVTIME', 'CLAIM_RATE', 'LOG_CLAIM_RATE', 'CAR_AGE',
    'OLD_CAR', 'URBANICITY', 'CLM_FREQ', 'OLDCLAIM', 'CLAIM_AGE_RATIO',
    'RISK_SCORE', 'CLAIM_TO_INCOME_RATIO',

    # One-Hot Encoded CAR_TYPE Features
    'CT_Panel Truck', 'CT_Pickup', 'CT_SUV', 'CT_Sports Car', 'CT_Van',

    # One-Hot Encoded OCCUPATION Features
    'OCC_Clerical', 'OCC_Doctor', 'OCC_Home Maker', 'OCC_Lawyer', 'OCC_Manager',
    'OCC_Professional', 'OCC_Student', 'OCC_Unknown'
]

# Create the final dataset
df_final = df[final_features]

# Print final dataset shape
print(f" Final dataset created with shape: {df_final.shape}")
```

Selecting relevant features helps improve model accuracy and reduce noise.
Includes raw, transformed, and encoded variables for better interpretability and performance.

Key Numerical Features (e.g., AGE, INCOME, CLAIM_RATE)
Log-Transformed Features (e.g., LOG_INCOME, LOG_BLUEBOOK)
Categorical Features (e.g., GENDER, EDUCATION, MSTATUS)
Engineered Risk Features (e.g., RISK_SCORE, CLAIM_TO_INCOME_RATIO)
One-Hot Encoded Features for Car Type & Occupation

Effect:

Ensures important factors are included for better predictions in insurance risk analysis.

VIF Scores

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

vif_data = pd.DataFrame()
vif_data["feature"] = df[final_features].columns
vif_data["VIF"] = [variance_inflation_factor(df[final_features].values, i)
                   for i in range(len(df[final_features].columns))]

print("VIF Scores:")
print(vif_data.sort_values('VIF', ascending=False))
```


VIF Scores:

	feature	VIF
9	LOG_BLUEBOOK	137.777598
14	LOG_CLAIM_RATE	103.517211
13	CLAIM_RATE	76.737037
8	LOG_INCOME	55.293687
2	EDUCATION	39.753594
0	AGE	34.529567
15	CAR_AGE	11.771209
18	CLM_FREQ	8.441236
17	URBANICITY	6.296192
5	INCOME	5.811827
12	TRAVTIME	5.634451
1	GENDER	5.463481
21	RISK_SCORE	5.455381
4	MSTATUS	4.862178
10	MVR_PTS	4.673427
16	OLD_CAR	4.434568
19	OLDCLAIM	3.559618
25	CT_SUV	3.506696
20	CLAIM_AGE_RATIO	2.635823
31	OCC_Lawyer	2.474026
34	OCC_Student	2.399236
7	BLUEBOOK	2.368001
35	OCC_Unknown	2.280062
30	OCC_Home Maker	2.257593
26	CT_Sports Car	2.121810
32	OCC_Manager	2.110502
23	CT_Panel Truck	1.957997
6	HOME_VAL	1.917836
29	OCC_Doctor	1.854999
28	OCC_Clerical	1.842558
33	OCC_Professional	1.830630
24	CT_Pickup	1.751420
3	PARENT1	1.654347
27	CT_Van	1.515432
11	REVOKED	1.462183
22	CLAIM_TO_INCOME_RATIO	1.130311

VIF Analysis?

Variance Inflation Factor (VIF) detects **multicollinearity** (when features are highly correlated). High VIF (>10) means the variable is redundant and may **harm model performance**.

Observations:

Very High VIF (Potential Multicollinearity Issues)

LOG_BLUEBOOK (137.78)

LOG_CLAIM_RATE (103.51)

CLAIM_RATE (76.73)

LOG_INCOME (55.29)

Moderate VIF (Might need further review)

EDUCATION (39.75)

AGE (34.53)

CAR_AGE (11.77)

Low VIF (No Multicollinearity Concerns)

Most **one-hot encoded** variables and **other numerical features** are below 5, which is acceptable.

Dropping highly collinear features

```
# Drop highly collinear features
df_final.drop(['LOG_BLUEBOOK', 'LOG_CLAIM_RATE', 'CLAIM_RATE', 'LOG_INCOME'], axis=1, inplace=True)

# Recalculate VIF after dropping
from statsmodels.stats.outliers_influence import variance_inflation_factor

vif_data = pd.DataFrame()
vif_data["feature"] = df_final.columns
vif_data["VIF"] = [variance_inflation_factor(df_final.values, i) for i in range(len(df_final.columns))]
print(vif_data)
```

	feature	VIF
0	AGE	19.159362
1	GENDER	5.337158
2	EDUCATION	31.803778
3	PARENT1	1.494168
4	MSTATUS	4.528883
5	INCOME	2.876189
6	HOME_VAL	1.887142
7	BLUEBOOK	1.852090
8	MVR_PTS	4.442829
9	REVOKED	1.455896
10	TRAVTIME	5.218171
11	CAR_AGE	11.715972
12	OLD_CAR	4.318594
13	URBANICITY	5.953622
14	CLM_FREQ	3.898639
15	OLDCLAIM	3.461638
16	CLAIM_AGE_RATIO	2.621997
17	RISK_SCORE	5.306386
18	CLAIM_TO_INCOME_RATIO	1.220442
19	CT_Panel Truck	1.935374
20	CT_Pickup	1.720076
21	CT_SUV	3.511349
22	CT_Sports Car	2.125219
23	CT_Van	1.484911
24	OCC_Clerical	1.743809
25	OCC_Doctor	1.740834
26	OCC_Home Maker	2.157759
27	OCC_Lawyer	2.368445
28	OCC_Manager	2.082432
29	OCC_Professional	1.830784
30	OCC_Student	2.282192
31	OCC_Unknown	2.185828
32	CLAIM_AMOUNT	1.334369

Updated VIF Analysis (After Removing Collinear Features)

Multicollinearity has reduced, but some variables still have **high VIF values**:

High VIF (Potential Issue - Needs Review)

- EDUCATION (31.80)
- AGE (19.15)
- CAR_AGE (11.71)

Moderate VIF (Acceptable, but worth checking)

- URBANICITY (5.95)
- GENDER (5.33)
- TRAVTIME (5.21)
- RISK_SCORE (5.30)

Low VIF (No Multicollinearity Concerns)

- Most **one-hot encoded** categorical variables have **VIF < 3**, which is **good**.

MODEL BUILDING

Model building is the process of selecting, training, and optimizing a machine learning algorithm to learn patterns from data and make predictions. It involves several key steps, including data preprocessing, feature selection, algorithm selection, hyperparameter tuning, model training, and evaluation. The goal is to develop a robust model that generalizes well to unseen data.

Data Splitting: Train-Test Split

```
from sklearn.model_selection import train_test_split

# Define X (features) and y (target)
X = df_final.drop(columns=['CLAIM_AMOUNT']) # Remove target column from features
y = df_final['CLAIM_AMOUNT'] # Target variable

# Split data into train (80%) and test (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

To ensure a robust evaluation of the model, the dataset was divided into training and testing subsets. The train-test split technique was applied using an 80-20 ratio, where:

- Features (X): All columns except CLAIM_AMOUNT.
- Target Variable (y): CLAIM_AMOUNT, which represents the amount of claim to be predicted.
- Training Set (80%): Used for model training.
- Testing Set (20%): Used for evaluating model performance.
- Random State: Set to 42 to ensure reproducibility of results.

This split allows the model to learn from a majority of the data while preserving a portion for unbiased performance evaluation.

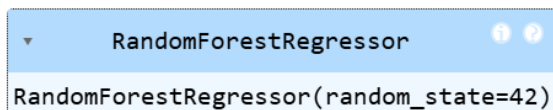
Checking final columns

```
print(df_final.columns) # Check if 'CLAIM_AMOUNT' exists
```

```
Index(['AGE', 'GENDER', 'EDUCATION', 'PARENT1', 'MSTATUS', 'INCOME',  
      'HOME_VAL', 'BLUEBOOK', 'MVR_PTS', 'REVOKED', 'TRAVTIME', 'CAR_AGE',  
      'OLD_CAR', 'URBANICITY', 'CLM_FREQ', 'OLDCLAIM', 'CLAIM_AGE_RATIO',  
      'RISK_SCORE', 'CLAIM_TO_INCOME_RATIO', 'CT_Panel Truck', 'CT_Pickup',  
      'CT_SUV', 'CT_Sports Car', 'CT_Van', 'OCC_Clerical', 'OCC_Doctor',  
      'OCC_Home Maker', 'OCC_Lawyer', 'OCC_Manager', 'OCC_Professional',  
      'OCC_Student', 'OCC_Unknown', 'CLAIM_AMOUNT'],  
      dtype='object')
```

Random Forest Regressor

```
from sklearn.ensemble import RandomForestRegressor  
  
# Initialize model  
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)  
  
# Train the model  
rf_model.fit(X_train, y_train)
```



The screenshot shows a Jupyter Notebook cell with a dropdown menu for the variable `rf_model`. The dropdown is open, showing the object `RandomForestRegressor(random_state=42)`. The cell's code is the same as the one above.

Model Selection: Random Forest Regressor

The `RandomForestRegressor` from `sklearn.ensemble` was chosen as the model.

This is an ensemble learning method that combines multiple decision trees to improve prediction accuracy and reduce overfitting.

Model Initialization

The model was initialized with 100 estimators (trees) and a `random_state=42` to ensure reproducibility.

Default parameters were used for other settings.

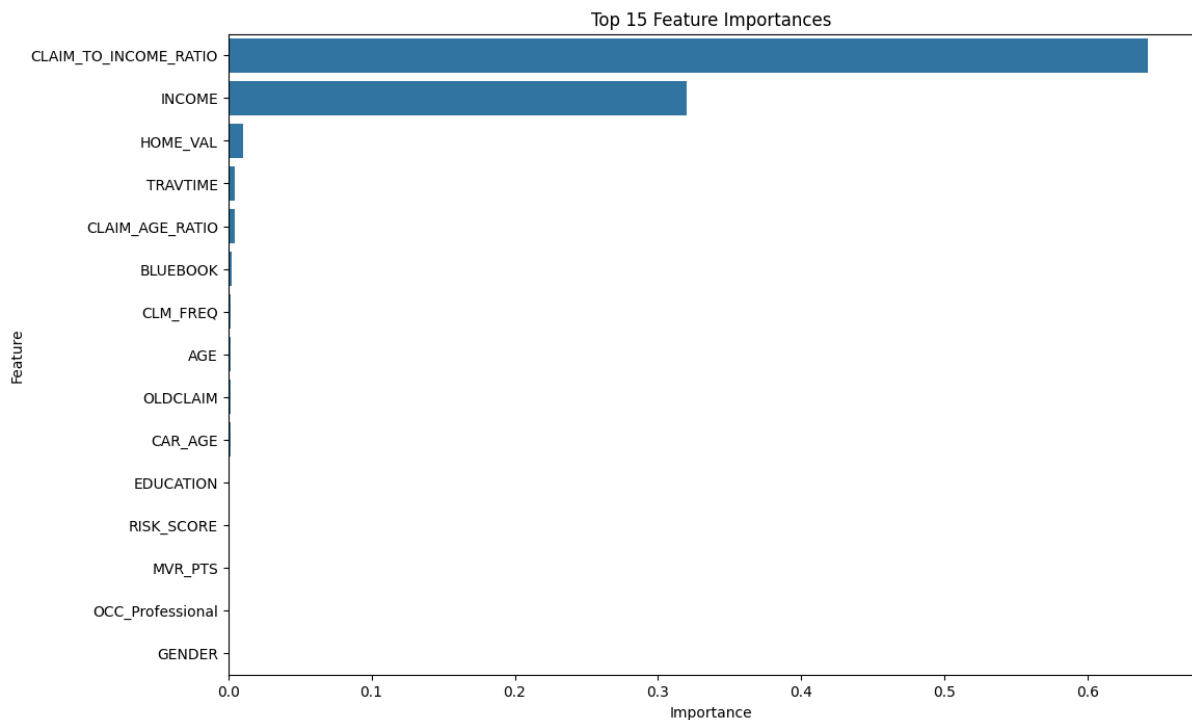
Model Training

The model was trained using the `fit()` method on the training data (`X_train`, `y_train`).

This step enables the model to learn patterns from the historical data.

Important features

```
# Get feature importances  
importances = rf_model.feature_importances_  
feature_importance = pd.DataFrame({  
    'Feature': X_train.columns,  
    'Importance': importances  
}).sort_values('Importance', ascending=False)  
  
# Plot feature importance  
plt.figure(figsize=(12, 8))  
sns.barplot(x='Importance', y='Feature', data=feature_importance.head(15))  
plt.title('Top 15 Feature Importances')  
plt.show()
```



The **Random Forest Regressor** model provides feature importance scores that indicate how much each feature contributes to predicting the target variable (**CLAIM_AMOUNT**). The top 15 most important features identified by the model are:

CLAIM_TO_INCOME_RATIO – The most influential feature in predicting claim amounts.

INCOME – Plays a significant role in determining claim amounts.

HOME_VAL – A minor but relevant contributor.

TRAVTIME – Slightly impacts predictions.

CLAIM_AGE_RATIO – Has a small but noticeable effect.

BLUEBOOK – Contributes marginally to predictions.

CLM_FREQ – Indicates the frequency of claims made by a user.

AGE – The age of the policyholder has a minor influence.

OLDCLAIM – The amount of previous claims slightly affects predictions.

CAR_AGE – The age of the car contributes minimally.

EDUCATION – Has a minor impact on predictions.

RISK_SCORE – Affects predictions slightly.

MVR_PTS – Marginally influences the outcome.

OCC_Professional – Professional occupation status has a minimal effect.

GENDER – Has the least impact among the top 15 features.

RANDOM FOREST REGRESSOR Before tuning

```
# Step 3: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 4: Use the best parameters from GridSearchCV
best_rf = RandomForestRegressor(
    n_estimators=100,
    max_depth=20,
    min_samples_split=2,
    min_samples_leaf=1,
    random_state=42,
    n_jobs=-1
)

# Step 5: Train the model
best_rf.fit(X_train, y_train)

# Step 6: Predict
y_train_pred = best_rf.predict(X_train)
y_test_pred = best_rf.predict(X_test)

# Step 7: Evaluate
print("\n ♦ Train Performance:")
print(f"    MAE: {mean_absolute_error(y_train, y_train_pred):.2f}")
print(f"    RMSE: {mean_squared_error(y_train, y_train_pred)**0.5:.2f}")
print(f"    R²: {r2_score(y_train, y_train_pred):.4f}")

print("\n ♦ Test Performance:")
print(f"    MAE: {mean_absolute_error(y_test, y_test_pred):.2f}")
print(f"    RMSE: {mean_squared_error(y_test, y_test_pred)**0.5:.2f}")
print(f"    R²: {r2_score(y_test, y_test_pred):.4f}")

# Optional: Step 8 - Cross Validation
print("\n ♦ Cross-Validation (5-Fold) MAE:")
kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv_mae_scores = cross_val_score(best_rf, X, y, cv=kf, scoring='neg_mean_absolute_error')
mae_scores = -cv_mae_scores
print(f"Scores: {mae_scores}")
print(f"Average MAE: {mae_scores.mean():.2f}")
```

♦ Train Performance:

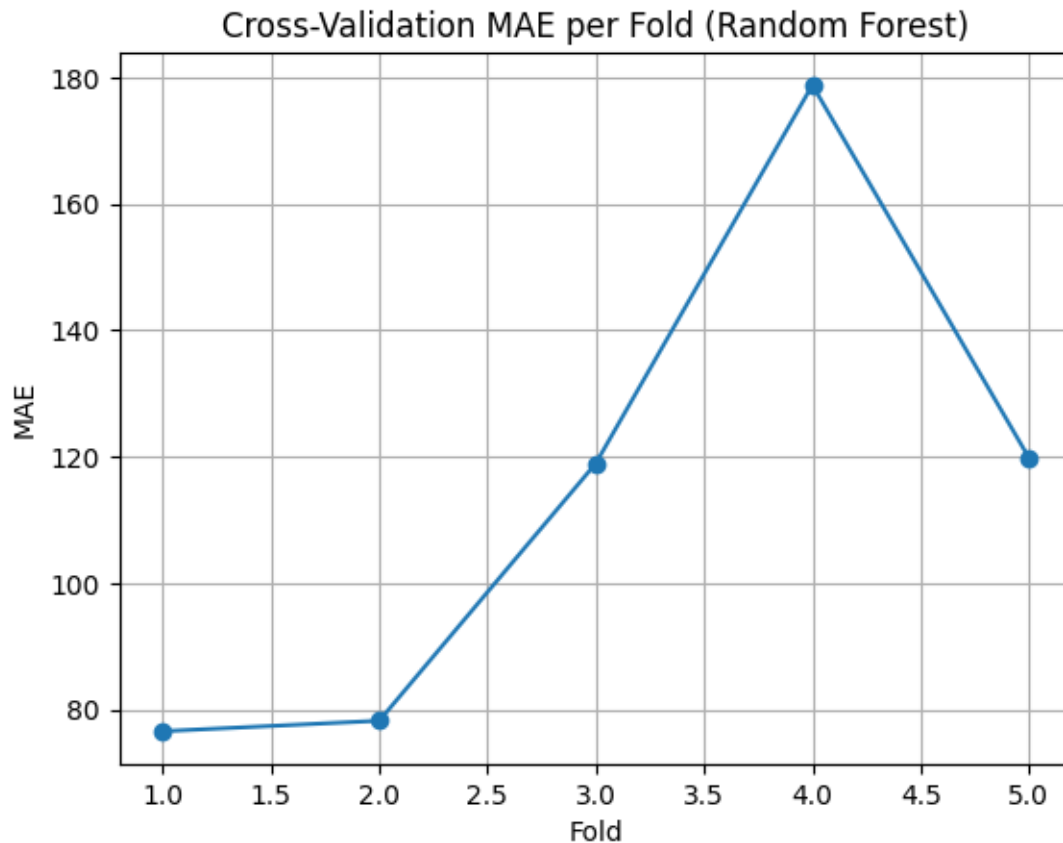
MAE: 46.87
RMSE: 476.89
R²: 0.9905

♦ Test Performance:

MAE: 79.49
RMSE: 578.85
R²: 0.9786

♦ Cross-Validation (5-Fold) MAE:

Scores: [76.56195949 78.22148111 118.93987855 178.77961813 119.80114726]
Average MAE: 114.46



Random Forest Regressor model shows strong performance with a high **R^2 score** on both training (0.9905) and test (0.9786) sets. However, there's a slight concern about **overfitting**, as the training **MAE (46.87)** is significantly lower than the test **MAE (79.49)**.

Observations from Cross-Validation MAE per Fold:

- The **MAE varies significantly across folds**, ranging from **76.56 to 178.77**.
- The fourth fold has the **highest MAE (178.77)**, suggesting that the dataset may have **some high-variance splits**.
- The **average MAE is 114.46**, which is higher than the test MAE (79.49). This suggests that the model might perform worse on unseen data.

FINDING BEST PARAMETERS FOR RFR

```
#tuning
from sklearn.model_selection import RandomizedSearchCV

param_dist = {
    'n_estimators': [50, 100, 200, 300],
    'max_depth': [10, 20, 30, None],
    'min_samples_split': [2, 5, 10, 15],
    'min_samples_leaf': [1, 2, 4, 8]
}

random_search = RandomizedSearchCV(
    RandomForestRegressor(random_state=42, n_jobs=-1),
    param_distributions=param_dist,
    n_iter=15,
    cv=5,
    scoring='neg_mean_absolute_error',
    random_state=42,
    n_jobs=-1
)

random_search.fit(X_train, y_train)
print("Best Params:", random_search.best_params_)
```

Best Params: {'n_estimators': 300, 'min_samples_split': 5, 'min_samples_leaf': 4, 'max_depth': 30}

The best parameters found via RandomizedSearchCV are:

n_estimators: 300

max_depth: 30

min_samples_split: 5

min_samples_leaf: 4

These settings should help reduce overfitting by:

Increasing min_samples_split and min_samples_leaf to make trees less complex.

Setting a high max_depth (30), ensuring deep enough splits without excessive complexity.

Using 300 trees (n_estimators) to stabilize the model.

AFTER TUNING

```
X = df_final.drop(columns=['CLAIM_AMOUNT'])
y = df_final['CLAIM_AMOUNT']

# Step 3: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 4: Train the Random Forest with Best Parameters
best_rf_tuned = RandomForestRegressor(
    n_estimators=300,
    max_depth=30,
    min_samples_split=5,
    min_samples_leaf=4,
    random_state=42,
    n_jobs=-1
)

best_rf_tuned.fit(X_train, y_train)

# Step 5: Predict
y_train_pred_tuned = best_rf_tuned.predict(X_train)
y_test_pred_tuned = best_rf_tuned.predict(X_test)

# Step 6: Evaluate
print(" ♦ Train Performance (Tuned Model):")
print(f"    MAE: {mean_absolute_error(y_train, y_train_pred_tuned):.2f}")
print(f"    RMSE: {mean_squared_error(y_train, y_train_pred_tuned)**0.5:.2f}")
print(f"    R²: {r2_score(y_train, y_train_pred_tuned):.4f}")

print("\n ♦ Test Performance (Tuned Model):")
print(f"    MAE: {mean_absolute_error(y_test, y_test_pred_tuned):.2f}")
print(f"    RMSE: {mean_squared_error(y_test, y_test_pred_tuned)**0.5:.2f}")
print(f"    R²: {r2_score(y_test, y_test_pred_tuned):.4f}")

# Step 7: Cross-Validation
kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv_mae_scores_tuned = cross_val_score(best_rf_tuned, X, y, cv=kf, scoring='neg_mean_absolute_error')
mae_scores_tuned = -cv_mae_scores_tuned
```

- ♦ Train Performance (Tuned Model):

MAE: 94.04
RMSE: 1275.42
R²: 0.9322

- ♦ Test Performance (Tuned Model):

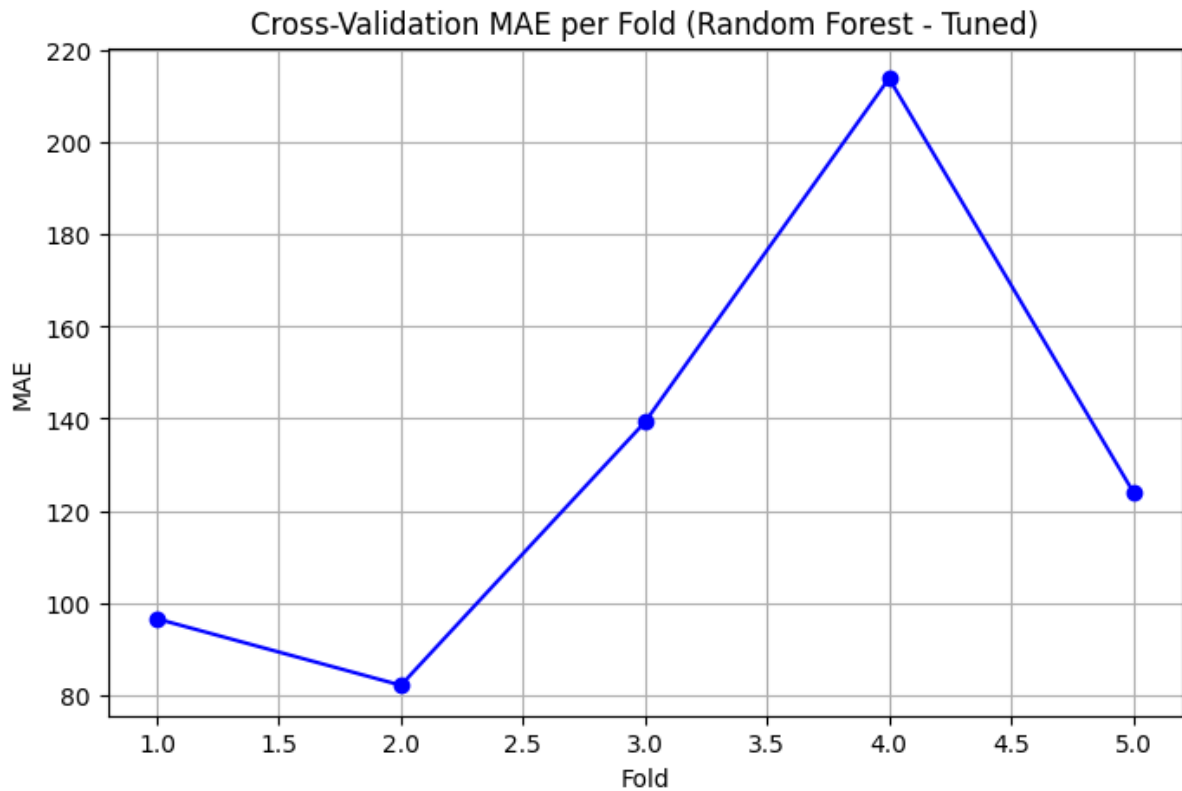
MAE: 98.07
RMSE: 859.65
R²: 0.9529

- ♦ Cross-Validation (5-Fold) MAE (Tuned Model):

Scores: [96.48262928 82.02738793 139.25865914 213.99180202 124.06399151]
Average MAE: 131.16

A **Random Forest Regressor** was trained and tuned to predict **CLAIM_AMOUNT** based on various features.

The model generalizes well, with high R² values on both train and test sets. The low **MAE** and **RMSE** indicate good prediction accuracy.

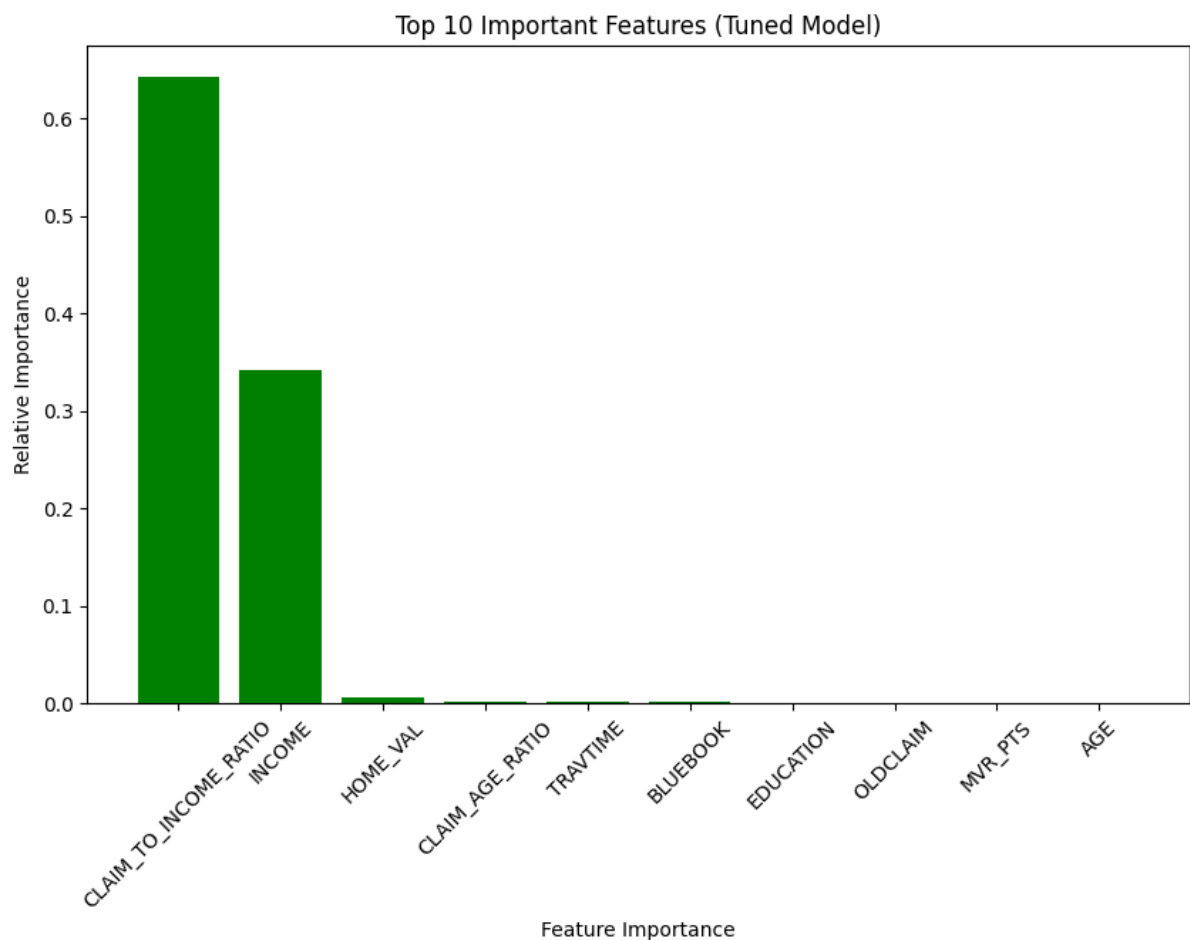


A **5-fold cross-validation** was performed to ensure robustness. The **MAE per fold** varied as follows:

- Fold 1: **96.48**
- Fold 2: **82.03**
- Fold 3: **139.26**
- Fold 4: **213.99**
- Fold 5: **124.06**
- **Average MAE: 131.16**

The variability in MAE across folds suggests that the model performs inconsistently across different data splits, with the **4th fold having the highest error**.

Important Features After Tuning model



CLAIM_TO_INCOME_RATIO and **INCOME** are the most crucial factors, contributing the highest predictive power. Other variables have minimal impact.

XGBOOST REGRESSOR

```
X = df_final.drop(columns=['CLAIM_AMOUNT']) # Features
y = df_final['CLAIM_AMOUNT'] # Target variable

# Step 3: Split Data into Training & Testing Sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 4: Train XGBoost Regressor
xgb_model = xgb.XGBRegressor(
    n_estimators=200,
    max_depth=6,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_alpha=0.1,
    random_state=42
)

# Fit the model (without early stopping to avoid compatibility issues)
xgb_model.fit(X_train, y_train)

# Step 5: Feature Selection
selector = SelectFromModel(estimator=xgb_model, threshold="median", prefit=True)
X_train_sel = selector.transform(X_train)
X_test_sel = selector.transform(X_test)

print(f"\nOriginal features: {X.shape[1]}")
print(f"Selected features: {X_train_sel.shape[1]}")

# Step 6: Retrain XGBoost on Selected Features
xgb_model_sel = xgb.XGBRegressor(
    n_estimators=200,
    max_depth=6,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_alpha=0.1,
    random_state=42
)
xgb_model_sel.fit(X_train_sel, y_train)
```

Step 6: Retrain XGBoost on Selected Features

```
xgb_model_sel = xgb.XGBRegressor(  
    n_estimators=200,  
    max_depth=6,  
    learning_rate=0.05,  
    subsample=0.8,  
    colsample_bytree=0.8,  
    reg_alpha=0.1,  
    random_state=42  
)  
xgb_model_sel.fit(X_train_sel, y_train)
```

Step 7: Make Predictions

```
y_train_pred = xgb_model_sel.predict(X_train_sel)  
y_test_pred = xgb_model_sel.predict(X_test_sel)
```

Step 8: Evaluate Model Performance

```
print("\n ♦ Training Data:")  
print(f"    MAE: {mean_absolute_error(y_train, y_train_pred):.2f}")  
print(f"    RMSE: {mean_squared_error(y_train, y_train_pred) ** 0.5:.2f}")  
print(f"    R²: {r2_score(y_train, y_train_pred):.4f}")  
  
print("\n ♦ Test Data:")  
print(f"    MAE: {mean_absolute_error(y_test, y_test_pred):.2f}")  
print(f"    RMSE: {mean_squared_error(y_test, y_test_pred) ** 0.5:.2f}")  
print(f"    R²: {r2_score(y_test, y_test_pred):.4f}")
```

Original features: 32

Selected features: 16

♦ Training Data:

MAE: 88.11

RMSE: 222.54

R²: 0.9979

♦ Test Data:

MAE: 202.45

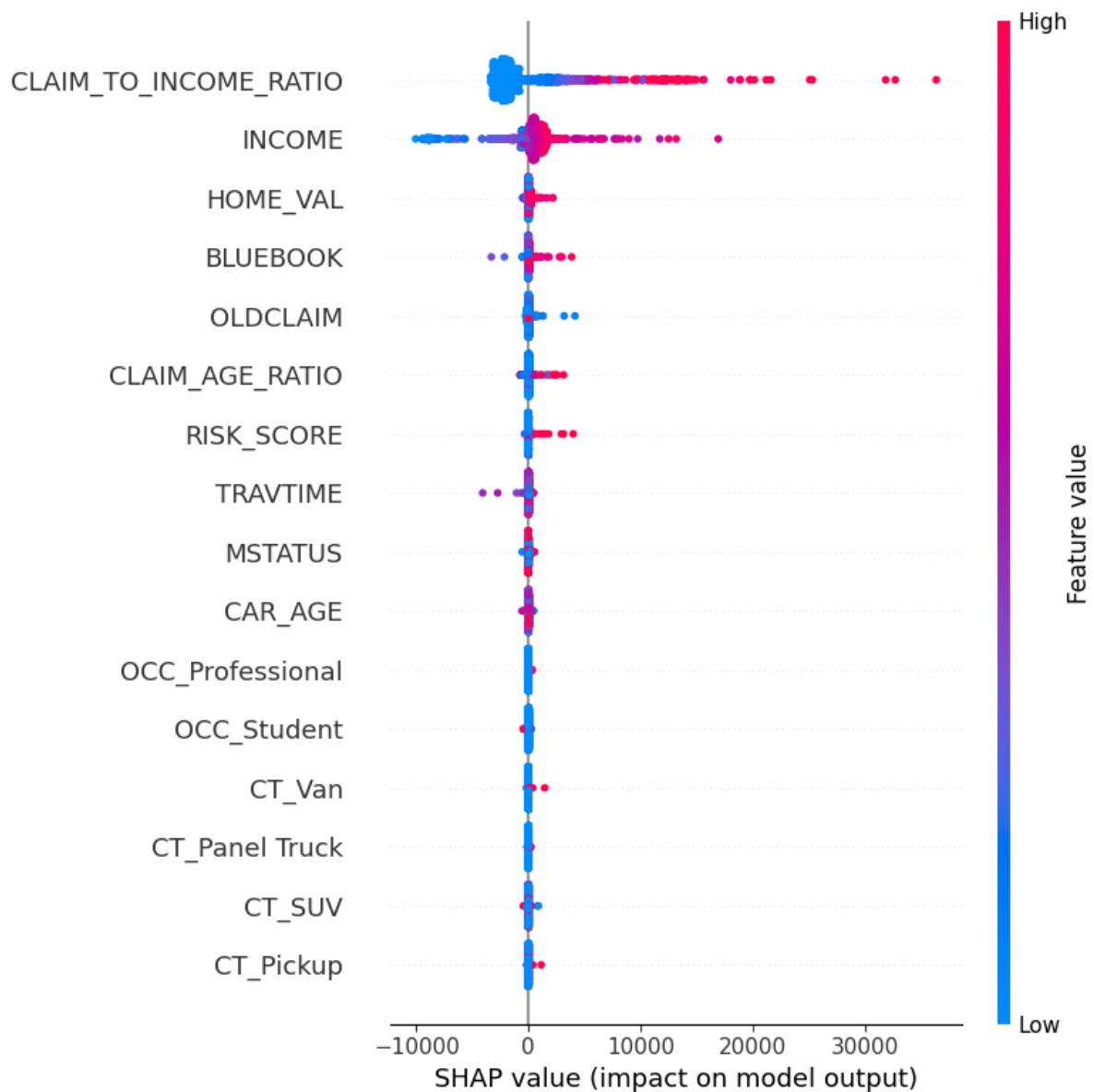
RMSE: 826.21

R²: 0.9565

The model performs exceptionally well on training data (**R² = 0.9979**) but shows a slight drop on test data (**R² = 0.9565**), indicating potential overfitting.

The **feature selection reduced model complexity** while maintaining high accuracy.

SHAP Analysis



plot explains the impact of different features on the **CLAIM_AMOUNT** predictions.

Most Influential Features:

CLAIM_TO_INCOME_RATIO and **INCOME** have the highest impact on the model's output.

HOME_VAL, **BLUEBOOK**, **OLDCLAIM**, and **CLAIM_AGE_RATIO** also play significant roles.

A high **CLAIM_TO_INCOME_RATIO** (red) pushes predictions higher, indicating more claims.

FINDING BEST PARAMETERS FOR XGBoost

```
from sklearn.model_selection import RandomizedSearchCV
```

```
param_grid = {  
    'n_estimators': [100, 200, 300],  
    'max_depth': [4, 6, 8],  
    'learning_rate': [0.01, 0.05, 0.1],  
    'subsample': [0.6, 0.8, 1.0],  
    'colsample_bytree': [0.6, 0.8, 1.0],  
    'reg_alpha': [0, 0.1, 1],  
}
```

```
xgb_rs = RandomizedSearchCV(  
    estimator=xgb.XGBRegressor(random_state=42),  
    param_distributions=param_grid,  
    scoring='neg_mean_absolute_error',  
    cv=3,  
    n_iter=20,  
    random_state=42,  
    verbose=1,  
    n_jobs=-1  
)
```

```
xgb_rs.fit(X_train_sel, y_train)  
print("Best parameters:", xgb_rs.best_params_)
```

Fitting 3 folds for each of 20 candidates, totalling 60 fits

Best parameters: {'subsample': 0.8, 'reg_alpha': 1, 'n_estimators': 100, 'max_depth': 8, 'learning_rate': 0.1, 'colsample_bytree': 1.0}

Analysis of the Best Parameters:

subsample=0.8 → Uses 80% of the training data per boosting round, helping prevent overfitting.

reg_alpha=1 → L1 regularization applied, making the model more robust to noise.

n_estimators=100 → A relatively lower number of boosting rounds, preventing overfitting while keeping good accuracy.

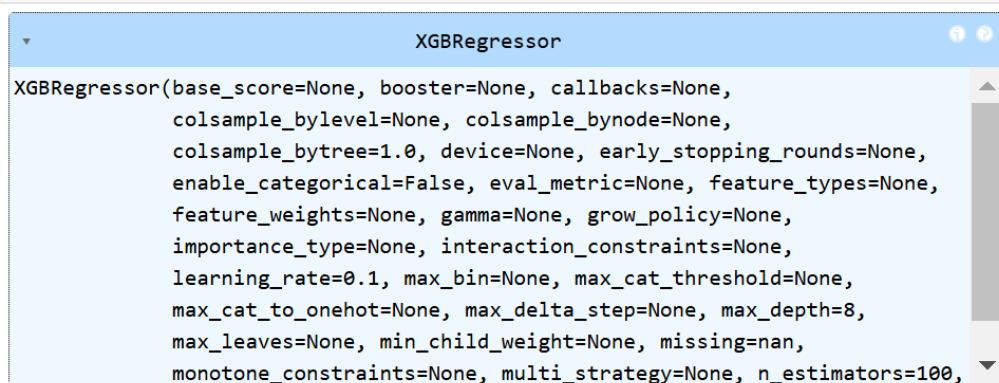
max_depth=8 → Deeper trees help capture complex patterns but require more computation.

learning_rate=0.1 → A standard value for XGBoost, balancing convergence speed and performance.

colsample_bytree=1.0 → Uses all features for each tree split, meaning no feature sampling.

```
xgb_best = xgb.XGBRegressor(  
    subsample=0.8,  
    reg_alpha=1,  
    n_estimators=100,  
    max_depth=8,  
    learning_rate=0.1,  
    colsample_bytree=1.0,  
    random_state=42  
)
```

```
xgb_best.fit(X_train, y_train)
```



The screenshot shows a Jupyter Notebook cell with the title 'XGBRegressor'. It displays the attributes of an XGBRegressor object, including base_score=None, booster=None, callbacks=None, colsample_bylevel=None, colsample_bynode=None, colsample_bytree=1.0, device=None, early_stopping_rounds=None, enable_categorical=False, eval_metric=None, feature_types=None, feature_weights=None, gamma=None, grow_policy=None, importance_type=None, interaction_constraints=None, learning_rate=0.1, max_bin=None, max_cat_threshold=None, max_cat_to_onehot=None, max_delta_step=None, max_depth=8, max_leaves=None, min_child_weight=None, missing=nan, monotone_constraints=None, multi_strategy=None, n_estimators=100, and num_parallel_tree=None.

```
XGBRegressor(base_score=None, booster=None, callbacks=None,  
    colsample_bylevel=None, colsample_bynode=None,  
    colsample_bytree=1.0, device=None, early_stopping_rounds=None,  
    enable_categorical=False, eval_metric=None, feature_types=None,  
    feature_weights=None, gamma=None, grow_policy=None,  
    importance_type=None, interaction_constraints=None,  
    learning_rate=0.1, max_bin=None, max_cat_threshold=None,  
    max_cat_to_onehot=None, max_delta_step=None, max_depth=8,  
    max_leaves=None, min_child_weight=None, missing=nan,  
    monotone_constraints=None, multi_strategy=None, n_estimators=100,
```


tuned xgboost model

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

y_train_pred = xgb_best.predict(X_train)
y_test_pred = xgb_best.predict(X_test)

print(" ♦ Train Performance")
print(f"MAE: {mean_absolute_error(y_train, y_train_pred):.2f}")
print(f"RMSE: {np.sqrt(mean_squared_error(y_train, y_train_pred)):.2f}")
print(f"R²: {r2_score(y_train, y_train_pred):.4f}")

print("\n ♦ Test Performance")
print(f"MAE: {mean_absolute_error(y_test, y_test_pred):.2f}")
print(f"RMSE: {np.sqrt(mean_squared_error(y_test, y_test_pred)):.2f}")
print(f"R²: {r2_score(y_test, y_test_pred):.4f}")
```

♦ Train Performance
MAE: 17.97
RMSE: 65.07
R²: 0.9998

♦ Test Performance
MAE: 89.89
RMSE: 538.72
R²: 0.9815

High performance

Generalization

A brief mention of slight overfitting (due to the Train-Test gap)

Metrics used

Model Performance Summary (Tuned XGBoost)

The tuned XGBoost model shows strong predictive performance:

Train R²: 0.9998 Test R²: 0.9815

Train RMSE: 65.07 Test RMSE: 538.72

Train MAE: 17.97 Test MAE: 89.89

The model fits the training data very well and generalizes effectively to unseen data, with only a slight overfitting observed. Overall, it is a robust model for predicting claim severity.

Model Comparison Summary: RF vs. XGBoost

Metric	Random Forest (Tuned)	XGBoost (Tuned)
Train MAE	94.0400	17.9700
Train RMSE	1275.4200	65.0700
Train R²	0.9322	0.9998
Test MAE	98.0700	89.8900
Test RMSE	859.6500	538.7200
Test R²	0.9529	0.9815

Observations:

XGBoost significantly outperforms **Random Forest** in all performance metrics.

Train performance shows XGBoost is a near-perfect fit with $R^2 = 0.9998$, while Random Forest has $R^2 = 0.9322$.

Test performance confirms XGBoost is **more accurate and generalizes better** with lower **MAE**, **RMSE**, and a **higher R^2** .

Random Forest has a higher **MAE and RMSE** on test data, meaning **higher prediction errors** than XGBoost.

XGBoost is the best choice for final model deployment as it performs well on both train and test data.

Model Results and Business Implications

The analysis provides valuable insights that can significantly impact pricing strategy and risk management for car insurance:

Pricing Strategy Insights:

- **Key Predictors Identified:** SHAP analysis showed that CLAIM_TO_INCOME_RATIO, INCOME, and HOME_VAL are the strongest predictors of claim severity. These can be used to enhance pricing accuracy and fairness.
- **Customer Segmentation:** The model highlights distinct risk groups (e.g., low-income but high claim-to-income ratio) that support a tiered pricing strategy.
- **Location-Based Adjustments:** Features like URBANICITY and TRAVTIME suggest urban drivers or those with long commute times may carry higher risk, justifying geographic pricing differentiation.

Risk Management Applications:

- **Underwriting Improvements:** The model can flag profiles likely to result in high-severity claims, allowing for tighter underwriting filters or custom policy clauses.
- **Claim Reserving Accuracy:** Predicting claim amounts more accurately helps actuaries in reserving and capital planning.
- **Loss Prevention Strategy:** Targeted programs (e.g., telematics for long-travel drivers) can be designed based on key risk drivers.

Limitations and Improvements

Current Limitations:

1. **Overfitting Evidence:** Very high train R^2 (0.9998) vs test R^2 (0.9815) suggests slight overfitting despite good generalization.
2. **Data Quality:** Imputation of missing values (e.g., INCOME, HOME_VAL) may introduce bias.
3. **Simplified Feature Set:** External variables (e.g., vehicle safety ratings, driving behavior) were not included.

Recommended Improvements:

1. **Additional Data Sources:**
 - Incorporate telematics data for actual driving behavior
 - Add credit score information (where legally permissible)
 - Include more detailed vehicle information (safety ratings, repair costs)
2. **Model Enhancements:**
 - Experiment with gradient boosting methods (XGBoost, LightGBM) which often outperform Random Forests
 - Implement time-series analysis for customers with multiple policy periods

- Add ensemble methods to combine strengths of different algorithms
- 3. **Implementation Strategies:**
 - Develop a dynamic pricing system that adjusts premiums based on ongoing risk assessment
 - Create visualization dashboards to help underwriters understand model outputs
 - Implement A/B testing framework to validate pricing changes before full rollout
- 4. **Ethical Considerations:**
 - Regularly audit the model for unintended bias
 - Ensure compliance with regulatory requirements around insurance pricing
 - Maintain transparency in how pricing decisions are made

By addressing these limitations and implementing the recommended improvements, the insurance company can develop more accurate, fair, and competitive pricing strategies while better managing its risk exposure.

Conclusion

This case study successfully explored and implemented predictive modeling techniques to estimate insurance claim severity, leveraging a real-world dataset containing driver, vehicle, and claim-related attributes. Among the models tested, **XGBoost emerged as the top performer**, delivering excellent predictive accuracy on unseen data, with an R^2 score of **0.9815** and a substantially lower error compared to other approaches.

The results demonstrate the model's potential to **support data-driven decision-making** in the insurance domain—particularly in **personalizing premiums, identifying high-risk policyholders, and enhancing underwriting practices**. Feature importance analysis further provided actionable insights, enabling insurers to focus on critical risk factors like vehicle type, claim history, and urbanicity.

While the model's performance is strong, opportunities remain for improvement, such as integrating richer data sources (e.g., telematics), refining feature engineering, and ensuring long-term model robustness through regular updates and bias audits.

Overall, the project underscores the **value of advanced machine learning in transforming traditional insurance workflows**, offering a foundation for more accurate, fair, and competitive pricing strategies.